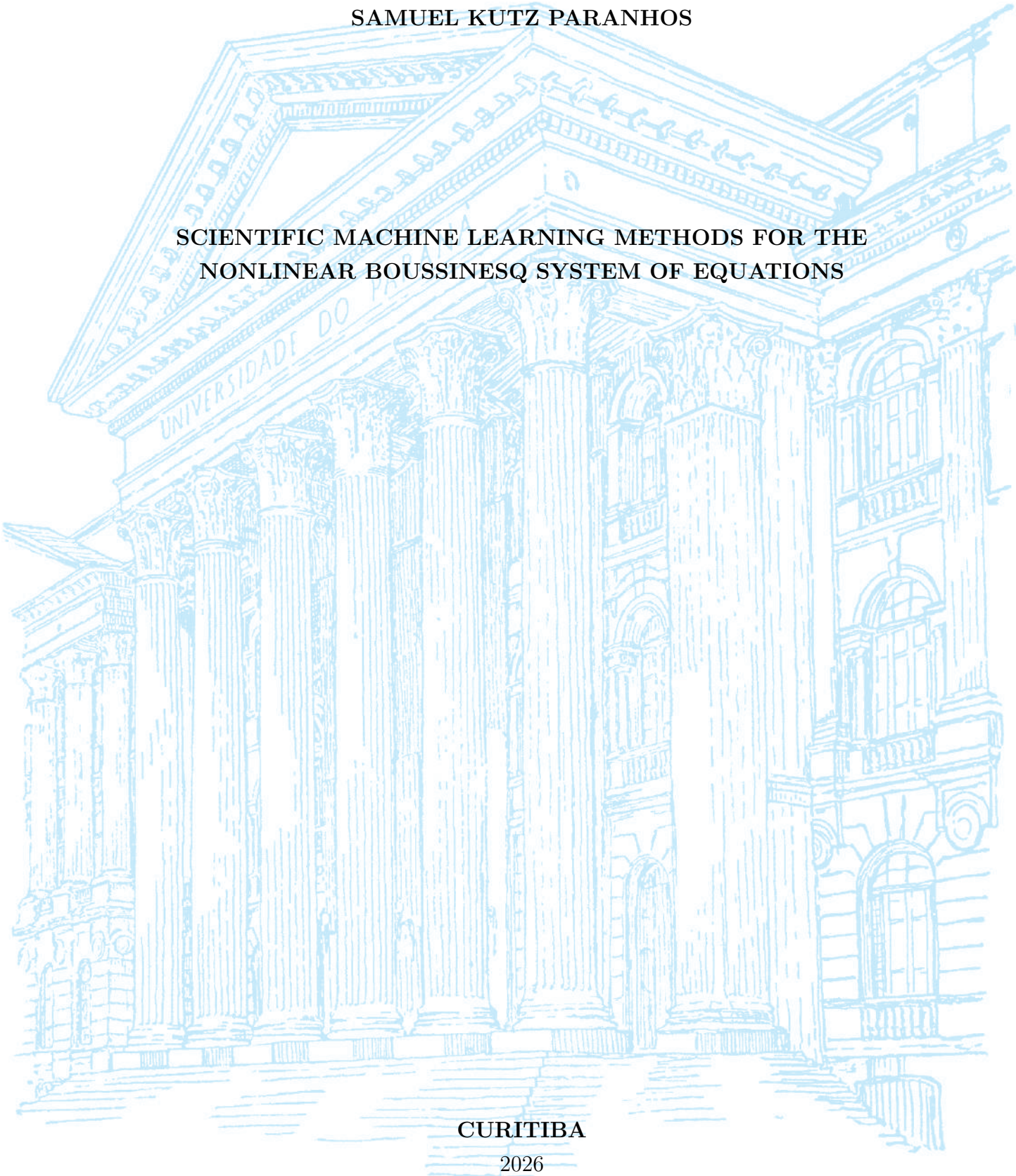
UNIVERSIDADE FEDERAL DO PARANÁ

SAMUEL KUTZ PARANHOS

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS



CURITIBA

2026

SAMUEL KUTZ PARANHOS

TRABALHO DE CONCLUSÃO DE CURSO

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS

Trabalho apresentado como requisito parcial à obtenção do grau de Bacharel em Matemática Industrial no Departamento de Matemática da Universidade Federal do Paraná.

Área de concentração: Matemática Industrial.

Orientador: Prof. Roberto Ribeiro.

Título do Projeto: Scientific Machine Learning Methods for the Nonlinear Boussinesq System of Equations.

CURITIBA

2026

RESUMO

Este trabalho investiga a eficácia de métodos de Scientific Machine Learning (SciML) na resolução e aproximação do sistema não linear de Boussinesq, um sistema de Equações Diferenciais Parciais (EDPs) utilizado para modelar a dinâmica de ondas aquáticas. Neste estudo, o sistema de Boussinesq será tratado como uma EDP dependente de dois parâmetros: os parâmetros de não linearidade e de dispersão. O objetivo é analisar o uso do ecossistema contemporâneo de ferramentas desenvolvido em torno do recente crescimento de Machine Learning como complemento aos métodos numéricos clássicos empregados na solução do sistema de Boussinesq. Foi utilizado o método pseudoespectral para gerar um conjunto de dados consistindo de soluções de referência do sistema de Boussinesq para diversos parâmetros e é então empregado como base de treinamento para três arquiteturas de SciML investigadas neste trabalho: Redes Neurais Informadas pela Física (PINNs), treinadas a um único parâmetro tanto com restrições de dados quanto de forma puramente física; Operadores Neurais de Fourier (FNO), operando em regime puramente supervisionado a vários parâmetros com uso exclusivo de dados sem nenhuma informação do operador diferencial da EDP; e Operadores Neurais Informados pela Física (PINO), que consiste em uma FNO com a inclusão do operador diferencial da EDP na função de perda da arquitetura. Como métricas de eficiência dessas arquiteturas, utilizamos a evolução do erro espectral e do erro relativo ao longo do espaço-tempo com relação a solução de referência. Como resultados, percebe-se que as PINNs tradicionais, especialmente quando treinadas sem o auxílio de dados, sofrem de um fenômeno conhecido como viés espectral, apresentando tendência a aprender primeiramente as componentes de baixa frequência da solução e grandes dificuldades em capturar detalhes essenciais de alta frequência à medida que a não linearidade aumenta. Em contrapartida, por aprenderem diretamente no espaço das frequências, os modelos de aprendizado de operadores, como o FNO e o PINO treinado com dados, atenuam substancialmente essa limitação nas baixas e médias frequências, embora nenhum método a elimine por completo: um erro residual de alta frequência persiste em todas as arquiteturas, e o viés reaparece de forma mais acentuada quando o PINO é treinado de maneira puramente física. Observa-se ainda que a inclusão do resíduo físico confere estabilidade temporal, suprimindo o artefato de Gibbs que degrada o FNO no instante final. Assim, concluímos que a inclusão de dados na função de perda constitui a estratégia mais eficaz para mitigar — ainda que não anular — o viés espectral comumente presente nessas arquiteturas.

Palavras-chave: Scientific Machine Learning; Equações Diferenciais Parciais; Sistema de Boussinesq; Redes Neurais Informadas pela Física.

ABSTRACT

This work investigates the effectiveness of Scientific Machine Learning (SciML) methods in solving and approximating the nonlinear Boussinesq system, a system of Partial Differential Equations (PDEs) used to model water wave dynamics. In this study, the Boussinesq system will be treated as a PDE dependent on two parameters: the nonlinearity and dispersion parameters. The objective is to analyze the use of the contemporary ecosystem of tools developed around the recent growth of Machine Learning as a complement to the classical numerical methods employed in solving the Boussinesq system. The pseudospectral method was used to generate a dataset consisting of reference solutions of the Boussinesq system for various parameters and is then employed as the training basis for three SciML architectures investigated in this work: Physics-Informed Neural Networks (PINNs), trained at a single parameter both with data constraints and purely physics-informed; Fourier Neural Operators (FNOs), operating in a purely supervised regime at multiple parameters using only data without any differential operator information from the PDE; and Physics-Informed Neural Operators (PINOs), which consist of an FNO with the inclusion of the PDE differential operator in the architecture loss function. As efficiency metrics for these architectures, we use the evolution of spectral error and relative error over space-time with respect to the reference solution. As results, it is observed that traditional PINNs, especially when trained without the aid of data, suffer from a phenomenon known as spectral bias, showing a tendency to learn the low-frequency components of the solution first and great difficulty capturing essential high-frequency details as nonlinearity increases. In contrast, by learning directly in the frequency space, operator learning models such as FNO and PINO trained with data attenuate this limitation substantially in the low and mid frequencies, although no method removes it entirely: a residual high-frequency error persists across all architectures, and the bias re-emerges more strongly when PINO is trained in a purely physics-informed manner. We further observe that the physics residual confers temporal stability, suppressing the Gibbs artifact that degrades the FNO at the final time. Thus, we conclude that including data in the loss function is the most effective strategy to mitigate — though not to eliminate — the spectral bias commonly present in these architectures.

Keywords: Scientific Machine Learning; Partial Differential Equations; Boussinesq System; Physics-Informed Neural Networks.

ACKNOWLEDGEMENTS

I would like to thank CNPq, the LabFluid laboratory, and the Federal University of Paraná (UFPR) for their support, infrastructure, and guidance during this research.

This work was supported by CNPq and developed with the collaboration of the LabFluid research group at UFPR.

CONTENTS

RESUMO 3
ABSTRACT 4

ACKNOWLEDGEMENTS **5**

LIST OF FIGURES **7**

LIST OF ABBREVIATIONS **10**

LIST OF SYMBOLS **11**

1 INTRODUCTION **13**
1.1 Literature Overview on Scientific Machine Learning for PDEs 13
1.2 The Boussinesq System of Equations 16
1.3 Research Goals 18
1.4 Work Structure 19

2 MATHEMATICAL BACKGROUND **20**
2.1 The Boussinesq System 20
2.1.1 Recovering the Wave Equation 21
2.2 Pseudospectral Method 22
2.2.1 Spatial Discretization and the Discrete Fourier Transform 23
2.2.2 Spectral Formulation of the Boussinesq System 23
2.2.3 Time Evolution via Runge-Kutta 4 25
2.2.4 Training Dataset 25
2.3 Neural Networks 26
2.3.1 Multilayer Perceptrons 26
2.3.2 Gradient Flow 27
2.3.3 Neural Tangent Kernel 28
2.3.4 Spectral Bias 30
2.3.5 Physics-Informed Neural Networks 35
2.3.6 Spectral Bias in Physics-Informed Neural Networks 36

2.4 Neural Operators 37
2.4.1 General Definition 37
2.4.2 Fourier Neural Operators 38
2.4.3 Physics-Informed Neural Operators 40

3 METHODOLOGY **42**
3.1 Experimental Setup and Reproducibility 42
3.2 Reference Dataset Generation 43
3.2.1 The One-Parameter Family 44
3.2.2 Discretization and Tensor Encoding 44
3.2.3 Normalization 45
3.3 Evaluation Protocol and Error Metrics 45
3.3.1 Metrics 46
3.4 Physics-Informed Neural Network Experiment 47
3.5 Fourier Neural Operator Experiment 47
3.6 Physics-Informed Neural Operator Experiment 48
3.7 Neural Tangent Kernel Diagnostic 49
3.8 Summary of Hyperparameters 49

4 NUMERICAL RESULTS **51**
4.1 Training Cost 51
4.2 The Mechanism of Spectral Bias: NTK Diagnostic 52
4.3 Spectral Bias Along Training 52
4.4 Spectral Fidelity at the Intermediate Regime 54
4.5 The Temporal-Boundary Artifact 58
4.6 Accuracy Across the Nonlinearity Axis 64
4.7 Zero-Shot Resolution Behaviour 64
4.8 Synthesis 66

5 CONCLUSION **68**
5.1 Answers to the Research Questions 68
5.2 Limitations and Threats to Validity 69
5.3 Future Work 70

LIST OF FIGURES

2.1 Initial condition $\eta(x,0) = A\,\mathrm{sech}^2(x)$ for $A = 1$. 21
2.2 Example of MLP architecture of input (x,t), with $L_{\mathrm{MLP}} = 3$ hidden layers and $N_{\mathrm{MLP}} = 6$ neurons per layer. 27

4.1 Neural Tangent Kernel diagnostic for the Boussinesq PINN residual at $\alpha = \beta = 3.21$, for widths 8, 128 and 512. 53
4.2 Relative spectral error by frequency band during training. In every panel the low band (blue) is learned first and to the lowest error, the mid band (purple) trails it, and the high band (red) is learned last and least — the signature of spectral bias. Note the mixed reference samples: operators are tracked at $\alpha = \beta = 0.1$, PINNs at $\alpha = \beta = 3.21$. 55
4.3 PINN (no data) spatial spectrum at $\alpha = \beta = 3.21$. Already at $t = 0$ the predicted spectrum undershoots the reference tail, and the relative error rises monotonically with wavenumber; the mean spectral error grows from 0.17 at $t = 0$ to 0.70 at $t = 30$. 56
4.4 PINN (with data) spatial spectrum at $\alpha = \beta = 3.21$. The low-wavenumber crests are tracked more closely than in Figure 4.3, but the high-wavenumber undershoot and the growth of the mean error in time ($0.17 \to 0.58 \to 0.68$) persist. 57
4.5 FNO spatial spectrum at $\alpha = \beta = 3.21$. Low and mid wavenumbers are tracked almost exactly (mean error 0.013 at $t = 0$), but the error migrates to the high modes and grows steadily in time, reaching a mean of 0.40 at $t = 30$. 59
4.6 PINO (with data) spatial spectrum at $\alpha = \beta = 3.21$. The prediction tracks the reference across the band and, unlike the FNO, the mean error remains bounded in time ($0.077 \to 0.13 \to 0.20$): the physics residual stabilizes the temporal evolution. 60
4.7 PINO (no data) spatial spectrum at $\alpha = \beta = 3.21$. Without the supervised term the low- and mid-band undershoot returns (mean error $0.11 \to 0.39 \to 0.33$), confirming that the physics residual alone does not overcome spectral bias. 61
4.8 FNO across the nonlinearity axis ($\alpha = \beta \in \{0.1, 3.21, 4.2\}$), evaluated at resolution 256. The predicted surfaces (left) capture the two counter-propagating pulses, but the time-resolved relative error (right) exhibits a pronounced spike at the final time $t = 30$: the temporal Gibbs artifact from treating time as a periodic axis. 62
4.9 PINO (with data) across the same nonlinearity axis and resolution. The predicted surfaces are visually indistinguishable from the FNO, but the terminal-time spike of Figure 4.8 is absent: the error rises only gently toward $t = 30$. The physics residual and initial-condition term act as a temporal regularizer. 63

4.10 PINO (with data) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3.21$. The operator produces a coherent field at every resolution, but the relative error against the pseudospectral reference *grows* with resolution (median error rising from about 0.07 at 128 to about 0.16 at 512): discretization-invariance of form is not accuracy-invariance. 65

LIST OF ABBREVIATIONS

SciML	Scientific Machine Learning
ML	Machine Learning
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
FNO	Fourier Neural Operator
PINO	Physics-Informed Neural Operator
MLP	Multilayer Perceptron
RK4	Runge–Kutta 4th-order method
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
NTK	Neural Tangent Kernel
MSE	Mean Squared Error
SGD	Stochastic Gradient Descent
AD	Automatic Differentiation

LIST OF SYMBOLS

Domain and general operators

x	spatial coordinate
t	temporal coordinate
$u(x,t)$	PDE solution (generic)
$\mathcal{D}$	differential operator of the PDE
$\mathcal{L}$	linear differential operator; also a loss functional
∇	gradient (nabla) operator
N_x	number of spatial grid points
N_t	number of time steps
Δx, Δt	spatial and temporal grid spacings

Boussinesq system and spectral quantities

$\eta(x,t)$	surface elevation field
$u(x,t)$	depth-averaged horizontal velocity field
α	nonlinearity parameter of the Boussinesq system
β	dispersion parameter of the Boussinesq system
A	initial wave amplitude
L	domain half-length, $\Omega = [-L, L]$
k	wavenumber (integer mode index)
ξ_k	wavenumber $\pi k/L$ on the domain $[-L, L]$
$\omega(k)$	angular frequency / dispersion relation
$\hat{f}(k)$	Fourier coefficient of f at mode k
$\hat{f}(k,t)$	time-evolving Fourier mode of f at wavenumber k

Neural networks and Neural Tangent Kernel

θ	trainable parameter vector of a neural network
u_θ	neural network approximation of the PDE solution
$h^{(l)}$	hidden-layer activation vector at layer l
σ	activation function
$W^{(l)}$, $b^{(l)}$	weight matrix and bias vector at layer l
N_{MLP}	number of neurons in a hidden layer
L_{MLP}	number of layers in the neural network
$K(\theta)$	empirical Neural Tangent Kernel, $K = JJ^{\mathsf{T}}$
λ_k	NTK eigenvalue associated with mode k
$\hat{e}(k,t)$	Fourier coefficient of the training error at mode k

Neural operators

$G^\dagger$, G_θ	target solution operator and its neural approximation
$\mathcal{K}_\ell$	integral operator of the ℓ-th FNO spectral layer
κ_ℓ	kernel function of the FNO integral operator
P	lifting operator of the FNO
Q	projection operator of the FNO
d_c	channel (width) dimension of the FNO
$R_{\ell,k}$	FNO spectral weight (discretized kernel) at layer ℓ, mode k
$k_{\max}$	FNO mode-truncation threshold

1 INTRODUCTION

Since the start of the scientific thinking, mathematics has been one of the key languages in which we can understand the world, and Partial Differential Equations (PDEs) are one of the main dialects of the mathematical language we use to model and to simplify nature's complexity.

When modelling these equations, we cannot always afford them to have simple closed-form solutions (that is, when they have solution at all), this exposes the need of obtaining approximated solutions coming from numerical methods for these PDEs, also giving the option to circumvent analytical complexity. This allows the extraction of predictions, comparisons with data, build simulations and so on. These classical numerical methods, widely studied for all types of differential equations, usually rely on discretizing the PDE's domain into grids and performing temporal steps.

With the recent ascension of Machine Learning (ML) methods and a whole ecosystem of tools built around it, a natural path is to bridge these two worlds and let ML augment the numerical solution of PDEs, taking advantage of the many software frameworks developed. This is one facet of Scientific Machine Learning (SciML), the broader effort to fuse physics-based, mechanistic models with data-driven methods. Within SciML, this work focuses on the numerical solution of PDEs and, given the vast literature around the topic, restricts its review of recent works to hydrodynamics equations.

1.1 Literature Overview on Scientific Machine Learning for PDEs

The neural network building blocks underlying all of these approaches trace a line beginning with the formal neuron of McCulloch et al. (1943) and Rosenblatt's perceptron (ROSENBLATT, 1958), the first trainable single-layer model. The key algorithmic advance that unlocked multi-layer training was the backpropagation algorithm (RUMELHART et al., 1986), making gradient-based optimization practical for deep architectures. Shortly after, the universal approximation theorem (CYBENKO, 1989; HORNIK et al., 1989) provided the theoretical guarantee for a multilayer perceptron (MLP) with a single hidden layer of sufficient width to be able to approximate any continuous function on a compact domain to arbitrary precision. This combination of practical and universal

approximation established the MLP as the backbone of modern deep learning. The term neural network itself reflects the biological metaphor at the heart of these models, in which McCulloch et al. (1943) drew an explicit analogy to the firing of biological neurons, and this vocabulary propagated through the field, causing MLPs to be routinely and interchangeably referred to as artificial neural networks (ANNs) or feedforward neural networks. The idea of "deep" in deep learning, popularized following the breakthrough results of LeCun et al. (2015), simply denotes an MLP with many hidden layers, a regime in which representational capacity grows substantially and where the backpropagation-based training paradigm proves to be exceptionally efficient.

These ML foundations become the basis of a scientific methodology once the methdos are coupled to the physical laws that govern a problem of interest. This is the premise of Scientific Machine Learning, a term consolidated by Baker et al. (2019) to name the discipline at the intersection of scientific computing and machine learning, and surveyed in breadth by Karniadakis et al. (2021). Its unifying goal is to combine the interpretability, generalization, and inductive biases of physics-based models with the flexibility and data-adaptivity of ML, rather than treating the network as a pure black box. The methods span several families. Some recover governing equations directly from data, as in the Sparse Identification of Nonlinear Dynamics (SINDy) (BRUNTON et al., 2016). Others embed differential-equation structure into the network itself, as in Neural Ordinary Differential Equations (neural ODEs) (CHEN, R. T. Q. et al., 2018) and Universal Differential Equations (UDEs), in which a trainable network stands in for the unknown terms of an otherwise mechanistic model (RACKAUCKAS et al., 2020). These tools have been carried across a wide range of domains, from global weather and climate forecasting (BONEV et al., 2023) to molecular dynamics, materials design, and turbulence modelling (KARNIADAKIS et al., 2021). Among all these directions, the one that concerns us here is the use of SciML to obtain numerical solutions of PDEs and, more specifically, of the hydrodynamics equations that govern water waves. Given the vastness of the literature, the remainder of this review is devoted to that subarea.

The contemporary landscape of SciML for PDEs was reshaped by one idea in particular, the Physics-Informed Neural Networks (PINNs) (RAISSI et al., 2019), that is, the embedding of the governing PDE residual directly into the training loss of a Neural Network, working as regularization term for not only fitting data, but also respecting the underlying physics. In hydrodynamics, there is a growing body of work has applied PINNs to: water flow (DE PINA et al., 2021, 2023), shallow-water dynamics on the sphere (BIHLO et al., 2022), and inverse problems in strongly nonlinear wave systems (JAGTAP et al., 2022). The consistent result that we can address from reviews (CUOMO et al., 2023) is that PINNs are most effective when observational data are sparse and the task

is parameter identification, namely inverse problems, which are known for usually being not well-posed problems. But, for a reliable precise forward simulation at scale, they do not yet rival classical solvers. In our vision, these approaches serve best to complement them.

As a natural extension from neural networks, there is a recent development in learning maps between infinite-dimensional function spaces, in which the theoretical foundation was popularized by T. Chen et al. (1995), who extended the universal approximation theorem to nonlinear operator, in other words, a shallow network can approximate any continuous operator between Banach spaces to arbitrary precision. Bringing this idea into practical terms, Lu et al. (2021) introduced DeepONet, the first deep architecture expressly designed for learning operators coming from differential equations, demonstrating it across a range of ODEs and PDEs. Building on this line of work, operator learning puts aside the single-instance limitation of PINNs by training a model to approximate the full mapping from parameters and initial data to the solution, so that at inference time a new solution costs only a forward pass for the network. Parallel to DeepONet, Fourier Neural Operator (FNO) (LI et al., 2021) is one of the most widely adopted instance of this idea, it parametrizes an integral kernel in Fourier space, achieving speed-ups over other neural operators and is discretization-independent, the so-called zero-shot superesolution. Subsequent works explored alternative bases like wavelets (GUPTA et al., 2021), but FNO was validated on many types of equations such as dispersive and soliton wave equations (PEI et al., 2022; ZHONG et al., 2023), was also caled to geophysical problems spanning regional ocean modeling, shallow-water emulation, and global weather forecasting (BONEV et al., 2023; LKHAGVASUREN et al., 2024; UCHIDA et al., 2025; PATEL et al., 2025; YU et al., 2025).

Following the idea for PINNs, The Physics-Informed Neural Operator (PINO) (LI et al., 2023a) combines operator learning with PDE-residual constraints, cutting the labelled-data requirement while preserving physical consistency. Applications to shallow-water systems (ROSOFSKY et al., 2023) confirmed that this hybrid regime inherits the generalization of FNO without its data hunger. Broad surveys of the field (TODO, 2023) identify these physics-data approaches working together as the most promising SciML direction, although highlighting turbulence modeling, scalability, and noisy-data robustness as open challenges.

A known limitation that cuts across all SciML architectures is the so-called spectral bias, the tendency of gradient-based optimizers to learn the low-frequency components of a solution faster than the high-frequency ones (RAHAMAN et al., 2019; XU et al., 2020). The theoretical account of this behavior came with the Neural Tangent Kernel (NTK) (JACOT et al., 2018), which showed that a sufficiently wide network trains, to

leading order, as a linear model governed by a kernel that stays essentially fixed during optimization, the so-called lazy-training regime. In this picture each mode of the solution is learned at a rate set by its associated NTK eigenvalue, so the wide disparity between the largest and smallest eigenvalues translates directly into the frequency imbalance seen during training (ARORA et al., 2019; CAO et al., 2021; BORDELON et al., 2020). Wang, Yu, et al. (2022) carried this analysis to PINNs, showing that the eigenvalue disparity between the PDE-residual and initial-condition loss components is what drives their spectral bias. This pathology is especially consequential for dispersive wave problems, where the solution energy is broadly distributed across wavenumbers. A recent systematic study (KHODAKARAMI et al., 2026) showed that second-order optimizers combined with spectrum-aware loss formulations substantially mitigate it.

As far as this review reveals, SciML methods have been applied to a broad class of shallow-water and dispersive wave equations. To the best of our knowledge, however, no study has systematically evaluated FNO, PINO, or PINNs on the parametric 1D Boussinesq system, where both the nonlinearity coefficient α and the dispersion coefficient β vary simultaneously. This gap motivates our effort to implement and evaluate theses models with the goal of exposing its details for the user aiming to bring together this new set of tools.

1.2 The Boussinesq System of Equations

For this work, we choose to implement SciML methods for a specific PDE originating from J. Boussinesq's 1872 effort to give a theoretical account of the solitary wave of translation observed by J. Scott Russell in a canal in 1834 (BOUSSINESQ, 1872). Russell had noticed that a hump of water, set in motion by a stopping barge, could travel for miles without changing shape, a phenomenon the linear wave theory of the time could not explain. Boussinesq showed that the stability of such a wave is the result of a precise balance between nonlinear steepening and frequency dispersion, two effects that are separately destabilizing but mutually compensating in the shallow-water, long-wave regime. The mathematical structure of this balance, and of dispersive wave theory in general, are treated in depth by Whitham (1974) and Debnath (2012).

The modern two-field form of the system, for the free-surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, was systematically derived and classified by Bona et al. (2002), who identified a four-parameter family of equivalent Boussinesq systems obtainable from the incompressible, irrotational Euler equations by asymptotic simplifications. Earlier, Peregrine (1967) had already written down the standard form of these equations for water of variable depth, establishing the notation and scaling that remain in widespread use. What makes the Boussinesq system particularly suitable as

a parametric testbed for this work is that changing the nonlinearity and dispersion coefficients continuously deforms the solution from the near-linear, wave-packet regime to the strongly nonlinear soliton regime, giving rise to a more interesting range of behaviors within a single mathematical framework.

The specific one-dimensional form of the system used in this work, with its parameterization of the nonlinearity coefficient α and the dispersion coefficient β, was adopted from Martins (2023) and Martins et al. (2024). In the dataset generated for this study α and β are held equal, defining a one-parameter family of problems whose difficulty grows. Namely, for small $\alpha = \beta$ the dynamics are nearly linear and the solution is well-captured by low-frequency modes, while for large $\alpha = \beta$ the strong coupling between nonlinearity and dispersion produces sharper, more energetically distributed wave profiles that are fundamentally harder to learn. This controlled variation is precisely what makes the setting useful for comparing SciML architectures, with the goal of observing how each method learns the target solution's frequency.

Classical numerical methods for the Boussinesq system rely on pseudospectral and finite-difference discretizations that exploit the periodic or nearly periodic spatial structure of the waves. Pseudospectral (**trefethen2000spectral**) schemes achieve spectral accuracy on the spatial derivatives and are therefore well suited to the dispersive character of the equations (ENGSIG-KARUP et al., 2012); this is the approach used to generate the reference dataset in this work. On the application side, operational coastal-engineering codes such as FUNWAVE (TODO, 2010) implement Boussinesq-type solvers for harbor and nearshore wave propagation, demonstrating the practical relevance of the model beyond the academic setting.

SciML methods have recently only been applied to the Boussinesq equation, not the specific Boussinesq System. Zheng et al. (2023) applied parameter-integrated PINNs with phase-domain decomposition to reconstruct two-dimensional Boussinesq dynamics including phase transitions and rogue-wave formation, identifying spectral bias as a key obstacle at high nonlinearity. Wang et al. (2024) trained a physics-informed network to infer velocity and pressure fields from surface-elevation data alone on a Boussinesq model, demonstrating the inverse-problem capabilities of the approach. Complementary studies on related nonlinear wave equations (ZHANG et al., 2024; KUMAR et al., 2025) have confirmed that purely data-driven and purely physics-driven networks each display characteristic failure modes at strong nonlinearity. Taken together, these results suggest that a systematic, parametric evaluation of FNO/PINO/PINN on the 1D Boussinesq system, which is the focus of this work, can be fruitful to understand better the interactions between the method's ease of traning and nonlinearities and dispersiveness.

1.3 Research Goals

This work implements and evaluates three SciML methods on the parametric 1D Boussinesq system: Physics-Informed Neural Networks (PINNs), Fourier Neural Operators (FNOs), and Physics-Informed Neural Operators (PINOs). All results are measured against a pseudospectral reference solver, which sets a high-accuracy baseline. The study is guided by the following questions.

- Accuracy relative to the pseudospectral baseline: how closely can each SciML method reproduce the reference solutions as nonlinearity and dispersion grow? Where does each method start to fail, and how does its error scale with the difficulty of the regime?
- Data-driven, physics-informed, and hybrid training: how does the presence or absence of supervised reference data affect the accuracy and training behavior of each method? Does adding a physics residual term improve generalization when data are scarce, or does it introduce new difficulties?
- Spectral bias across training regimes: how does spectral bias manifest in each architecture when the error spectrum is tracked band by band during training, and to what extent does the inclusion of data or physics constraints alter the frequency at which errors concentrate?
- The mechanism behind spectral bias: does Neural Tangent Kernel (NTK) theory genuinely predict the frequency imbalance seen on the Boussinesq system, or only describe it after the fact? We put the theory to two quantitative tests on the PINN residual, across a range of network widths. The first targets the frozen-kernel (lazy-training) assumption the analysis depends on: we measure how far the network parameters and the empirical kernel drift over training, and whether the kernel's eigenvalue spectrum is essentially unchanged from initialization to convergence, as lazy training requires. The second targets the predicted driver of the bias: an eigenvalue spectrum that decays across many orders of magnitude, so that the modes carrying the high-frequency detail are learned far more slowly than the smooth, low-frequency ones. Confirming both would tie the frequency-band errors seen during training to a concrete mechanism rather than a loose analogy.

Together, these questions aim to provide a clear picture of the strengths and limitations of each SciML approach for dispersive, nonlinear wave problems, and to offer practical guidance for choosing among them.

1.4 Work Structure

This work is written with the goal of sharing SciML for senior undergraduates in mathematics, physics, or engineering. We recommend the reader to have some knowledge of scientific computing and linear algebra. The remaining chapters are organized as follows.

- Chapter 2 develops the mathematical background. It covers the Boussinesq system and its analytical properties, the pseudospectral solver, and the theoretical foundations of MLP, PINN, FNO, and PINO, including a discussion of spectral bias.
- Chapter 3 describes the experimental methodology. It details the dataset generation procedure, the model architectures, the training setup, and the evaluation metrics used throughout.
- Chapter 4 presents the numerical results and discusses the performance of each method across parameter regimes and spatial resolutions.
- Chapter 5 summarizes the main findings and outlines directions for future work.

2 MATHEMATICAL BACKGROUND

This chapter introduces the mathematical foundations of the methods studied in this work. Section 2.1 presents the Boussinesq system and the limiting wave-equation regime. Section 2.2 develops the pseudospectral numerical method used to generate reference solutions. Sections 2.3 and 2.4 introduce, respectively, the neural network and neural operator architectures studied in this work.

2.1 The Boussinesq System

The Boussinesq system is a model for long nonlinear dispersive waves in shallow water, governing the coupled evolution of two scalar fields: the free surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, both depending on position x and time t. In the one-dimensional, spatially periodic setting studied in this work, the full system reads:

$$\begin{cases} \eta_t + u_x + \alpha\,(\eta u)_x = 0, \\ u_t - \dfrac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0, \\ \eta(x,0) = A\ \mathrm{sech}^2(x), \quad u(x,0) = 0, \quad x \in [-L, L], \end{cases} \tag{2.1}$$

where $\alpha \geq 0$ is the nonlinearity parameter, $\beta > 0$ is the dispersion parameter, $A > 0$ is the initial amplitude, $L > 0$ is the domain half-length, and $\mathrm{sech}(z) = 2/(e^z + e^{-z})$ is the hyperbolic secant. The initial condition places a solitary-wave-like crest at the center $x = 0$ with zero initial velocity. This nondimensional form follows equation (3.1) of Martins (2023) (see also Martins et al. (2024)).

We note that $uu_x = \frac{1}{2}(u^2)_x$, so the momentum equation in (2.1) can equivalently be written with the conservative flux $\frac{\alpha}{2}(u^2)_x$ in place of $\alpha\,uu_x$. Both forms appear in the implementations: the PINN residual uses uu_x directly via automatic differentiation, while the pseudospectral solver exploits $\frac{1}{2}(u^2)_x$ for the physical-space product, as detailed in Section 2.2.

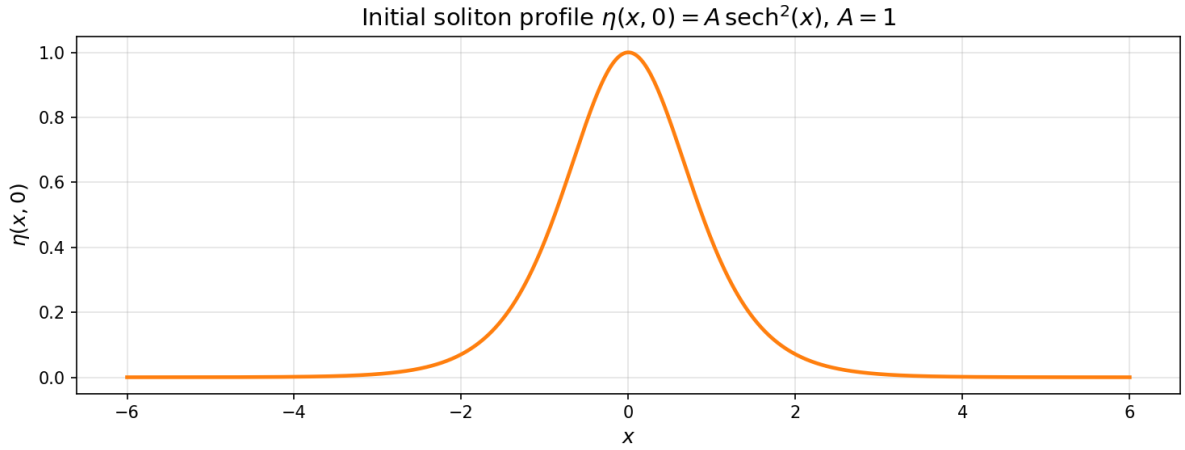


Figure 2.1 – Initial condition $\eta(x,0) = A\,\mathrm{sech}^2(x)$ for $A = 1$.

Source: The author (2026).

2.1.1 Recovering the Wave Equation

In the doubly linear limit $\alpha = \beta = 0$, all nonlinear and dispersive terms vanish and the Boussinesq system reduces to the symmetric hyperbolic system:

$$\eta_t + u_x = 0, \tag{2.2}$$

$$u_t + \eta_x = 0. \tag{2.3}$$

To obtain a single scalar equation, differentiate (2.2) with respect to t:

$$\eta_{tt} + u_{xt} = 0. \tag{2.4}$$

Differentiate (2.3) with respect to x:

$$u_{tx} + \eta_{xx} = 0. \tag{2.5}$$

Since $u_{xt} = u_{tx}$, subtracting the second from the first yields the wave equation for η:

$$\eta_{tt} = \eta_{xx}, \tag{2.6}$$

and the reverse procedure gives the same equation for u.

D'Alembert's formula (DEBNATH, 2012) provides the general solution to (2.6). Given initial data

$$\eta(x,0) = f(x), \qquad \eta_t(x,0) = g(x), \tag{2.7}$$

the solution is:

$$\eta(x,t) = \frac{f(x+t) + f(x-t)}{2} + \frac{1}{2}\int_{x-t}^{x+t} g(s)\,ds. \tag{2.8}$$

We now apply this to the initial conditions (2.1). From (2.2), $\eta_t(x,0) = -u_x(x,0) = 0$, so:

$$f(x) = A\,\mathrm{sech}^2(x), \qquad g(x) = 0. \tag{2.9}$$

Since $g \equiv 0$, the integral in (2.8) vanishes and the surface elevation evolves as:

$$\eta(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x+t) + \mathrm{sech}^2(x-t)\big]. \tag{2.10}$$

For u, from (2.3) we have $u_t(x,0) = -\eta_x(x,0)$, giving:

$$f(x) = 0, \qquad g(x) = -\eta_x(x,0). \tag{2.11}$$

The $f = 0$ term vanishes in (2.8), leaving:

$$u(x,t) = \frac{1}{2}\int_{x-t}^{x+t} \big(-\eta_x(s,0)\big)\,ds = -\frac{1}{2}\big[\eta(x+t,0) - \eta(x-t,0)\big]. \tag{2.12}$$

Substituting $\eta(x,0) = A\,\mathrm{sech}^2(x)$:

$$u(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x-t) - \mathrm{sech}^2(x+t)\big]. \tag{2.13}$$

The initial bump of amplitude A splits into two counter-propagating pulses of amplitude $A/2$, traveling at speed ± 1 without changing shape. Any deviation from this wave-equation baseline in the full system (2.1) is attributable either to dispersion ($\beta > 0$) or to nonlinearity ($\alpha > 0$).

2.2 Pseudospectral Method

The pseudospectral method exploits two complementary representations of the solution: Fourier space, where derivatives become exact algebraic multiplications, and physical space, where pointwise nonlinear operations are inexpensive. By alternating between the two representations as needed, the method attains spectral accuracy in space while handling the nonlinear coupling efficiently.

2.2.1 Spatial Discretization and the Discrete Fourier Transform

The spatial domain $\Omega = [-L, L]$ of total length $2L$ is discretized into N_x equispaced points:

$$x_j = -L + j\,\Delta x, \qquad \Delta x = \frac{2L}{N_x}, \qquad j = 0, \ldots, N_x - 1. \tag{2.14}$$

The Discrete Fourier Transform (DFT) of a function f sampled at these points is (TREFETHEN, 2000):

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j)\, e^{-2\pi i k j / N_x}, \qquad k = -\tfrac{N_x}{2}, \ldots, \tfrac{N_x}{2} - 1, \tag{2.15}$$

with its inverse recovering the physical values:

$$f(x_j) = \frac{1}{N_x} \sum_{k} \hat{f}(k)\, e^{2\pi i k j / N_x}. \tag{2.16}$$

The fundamental identity of the DFT with respect to differentiation is that, for a periodic function, the spatial derivative transforms into a pointwise multiplication (TREFETHEN, 2000; ENGSIG-KARUP et al., 2012):

$$\widehat{(f_x)}(k) = i\frac{\pi k}{L}\,\hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\frac{\pi^2 k^2}{L^2}\,\hat{f}(k). \tag{2.17}$$

This is in contrast to finite-difference methods, where derivatives are approximated with truncation error; in the DFT, equation (2.17) is exact for periodic functions.

We define the wavenumber of mode k on $[-L, L]$ as:

$$\xi_k \;:=\; \frac{\pi k}{L}. \tag{2.18}$$

With this definition, the derivative identities read:

$$\widehat{(f_x)}(k) = i\,\xi_k\,\hat{f}(k), \qquad \widehat{(f_{xx})}(k) = -\xi_k^2\,\hat{f}(k). \tag{2.19}$$

2.2.2 Spectral Formulation of the Boussinesq System

To bring the Boussinesq system (2.1) into the Fourier domain (MARTINS, 2023), we apply the DFT to each equation and use the derivative identities (2.19). The first equation, treating $\widehat{(\eta u)}(k,t)$ as a given nonlinear term, becomes:

$$\hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k,t). \tag{2.20}$$

The second equation requires more care because of the u_{xxt} term. Since spatial derivatives commute with the DFT:

$$\widehat{(u_{xxt})}(k,t) = -\xi_k^2\,\hat{u}_t(k,t). \tag{2.21}$$

Substituting into the DFT of the momentum equation in (2.1):

$$\hat{u}_t(k,t) + \frac{\beta}{3}\,\xi_k^2\,\hat{u}_t(k,t) + i\,\xi_k\,\hat{\eta}(k,t) + \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t) = 0. \tag{2.22}$$

Collecting the $\hat{u}_t(k,t)$ terms on the left and solving algebraically:

$$\hat{u}_t(k,t) = \frac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \tag{2.23}$$

The operator $1-\frac{\beta}{3}\frac{\partial^2}{\partial x^2}$, which requires a linear solve in physical space, becomes the diagonal scalar $1 + \frac{\beta\,\xi_k^2}{3}$ in Fourier space, reducing its inversion to a single division per mode.

Equations (2.20) and (2.23) can be written jointly as a first-order ODE system in time for each fixed wavenumber $k \in \mathbb{Z}$:

$$\begin{cases} \hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\,i\,\xi_k\,\widehat{(\eta u)}(k,t), \\[2ex] \hat{u}_t(k,t) = \dfrac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \end{cases} \tag{2.24}$$

The PDE system (2.1), two equations in (x,t), is thus equivalent to an infinite family of two-dimensional ODE systems (2.24), one for each $k \in \mathbb{Z}$, coupled through the nonlinear terms.

The nonlinear terms $\widehat{(\eta u)}(k,t)$ and $\widehat{(\frac{1}{2}u^2)}(k,t)$ cannot be computed directly in Fourier space without a convolution sum, which costs $O(N_x^2)$. The pseudospectral approach avoids this by computing products in physical space. For $\widehat{(\eta u)}(k,t)$, the procedure is:

1. recover $\eta(x_j)$ and $u(x_j)$ by applying the inverse DFT (2.16);
2. compute the product $(\eta u)_j = \eta(x_j) \cdot u(x_j)$ pointwise on the grid;
3. apply the DFT (2.15) to obtain $\widehat{(\eta u)}(k,t)$.

Similarly, $\widehat{(\frac{1}{2}u^2)}(k,t)$ is obtained by the same three-step procedure with $u^2(x_j) = u(x_j)^2$, where $u(x_j) = \mathrm{IDFT}(\hat{u})(x_j)$; and $(\eta u)(x_j) = \mathrm{IDFT}(\hat{\eta})(x_j) \cdot \mathrm{IDFT}(\hat{u})(x_j)$. Since each DFT/IDFT costs $O(N_x \log N_x)$ via the Fast Fourier Transform (FFT), this pseudospectral strategy assembles each nonlinear term in $O(N_x \log N_x)$ rather than $O(N_x^2)$.

2.2.3 Time Evolution via Runge-Kutta 4

We define the pseudospectral right-hand side operator F as the map

$$F(\hat{\eta}, \hat{u})(k) = \begin{pmatrix} -i\,\xi_k\,\hat{u}(k) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k) \\ \dfrac{-i\,\xi_k\,\hat{\eta}(k) - \alpha\, i\,\xi_k\big(\widehat{\frac{1}{2}u^2}\big)(k)}{1 + \dfrac{\beta\,\xi_k^2}{3}} \end{pmatrix}, \tag{2.25}$$

so that $F(\hat{\eta}, \hat{u}) = \big(\hat{\eta}_t,\, \hat{u}_t\big)$ as given by equations (2.20)–(2.23), with nonlinear terms assembled by the pseudospectral procedure above. Time evolution uses the classical fourth-order Runge-Kutta (RK4) scheme (SÜLI et al., 2003; BURDEN et al., 2011). Given the spectral state $y_n = (\hat{\eta}^n, \hat{u}^n)$ at time t_n, the four stage evaluations $\kappa_1, \kappa_2, \kappa_3, \kappa_4$ and the update to $t_{n+1} = t_n + \Delta t$ are:

$$\begin{aligned} \kappa_1 &= F(y_n), \\ \kappa_2 &= F\big(y_n + \tfrac{\Delta t}{2}\,\kappa_1\big)\,, \\ \kappa_3 &= F\big(y_n + \tfrac{\Delta t}{2}\,\kappa_2\big)\,, \\ \kappa_4 &= F(y_n + \Delta t\,\kappa_3)\,, \\ y_{n+1} &= y_n + \frac{\Delta t}{6}\,(\kappa_1 + 2\,\kappa_2 + 2\,\kappa_3 + \kappa_4). \end{aligned} \tag{2.26}$$

At each stage, the intermediate state is decoded to physical space to assemble the nonlinear products, then encoded back to Fourier space to evaluate the spectral derivatives. This split between physical-space products and Fourier-space derivatives is the defining operation of the pseudospectral method.

2.2.4 Training Dataset

The numerical experiments in this work vary only the PDE parameters (α, β) while keeping the initial condition fixed at (2.1). The models are therefore trained and evaluated on the dataset

$$\mathcal{S} = \big\{\big((\alpha_i, \beta_i),\, (\eta_i, u_i)\big)\big\}_{i=1}^{M}, \tag{2.27}$$

where (α_i, β_i) are the nonlinearity and dispersion parameters for the i-th sample, and (η_i, u_i) is the corresponding space-time solution obtained from the pseudospectral solver of Section 2.2 with those parameters and the shared initial condition (2.1).

2.3 Neural Networks

This section introduces the neural network architectures central to this work together with the theory that explains their training behavior. We begin with the Multilayer Perceptron as the building block, develop the Neural Tangent Kernel framework and the spectral bias it predicts for a network fitting data on a regular grid, and then turn to Physics-Informed Neural Networks, showing how the same kernel analysis extends to the physics-informed setting where spectral bias is a well-documented obstacle.

2.3.1 Multilayer Perceptrons

In this work, we focus on Multilayer Perceptrons (MLPs), popularized by Rumelhart et al. (1986). These networks are compositions of affine transformations parametrized by tunable weights and biases, separated by nonlinear activations, where an optimization process fits the network to a dataset.

Given a labeled dataset $\{(x_i, u_i)\}_{i=1}^{N}$, the objective is to fit the function $u_\theta(x)$ by minimizing a loss function, here the Mean Squared Error (MSE). With $\|\cdot\|$ denoting the Euclidean norm, the unconstrained optimization problem is:

$$\mathcal{L}_{\text{MLP}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|u_\theta(x_i) - u_i\|^2. \tag{2.28}$$

The MLP $u_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is defined by the composition:

$$h^{(0)} = x \in \mathbb{R}^{d_{\text{in}}}, \tag{2.29}$$

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right) \in \mathbb{R}^{N_{\text{MLP}}}, \quad l = 1, \ldots, L_{\text{MLP}} - 1, \tag{2.30}$$

$$u_\theta(x) = W^{(L_{\text{MLP}})} h^{(L_{\text{MLP}}-1)} + b^{(L_{\text{MLP}})} \in \mathbb{R}^{d_{\text{out}}}. \tag{2.31}$$

where:

- $L_{\text{MLP}} \in \mathbb{N}$ is the number of layers and $N_{\text{MLP}} \in \mathbb{N}$ the number of neurons per hidden layer;
- $W^{(l)} \in \mathbb{R}^{N_{\text{MLP}} \times N_{\text{MLP}}}$ and $b^{(l)} \in \mathbb{R}^{N_{\text{MLP}}}$ are the weight matrix and bias vector at layer l, with $W^{(1)} \in \mathbb{R}^{d_{\text{in}} \times N_{\text{MLP}}}$ and $W^{(L_{\text{MLP}})} \in \mathbb{R}^{N_{\text{MLP}} \times d_{\text{out}}}$ adapted at the boundaries;
- $\sigma : \mathbb{R} \to \mathbb{R}$ is a sigmoidal activation function applied elementwise, satisfying $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$;
- $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L_{\text{MLP}}}$ denotes all trainable parameters.

If the activation function is sigmoidal, MLPs are universal approximators: they can approximate any continuous function on a compact domain to arbitrary precision (CYBENKO, 1989).

In Figure 2.2, we show a common depiction of the neural network. Here the input vector is the pair $(x, t) \in \mathbb{R}^2$ and the output is $(\eta_\theta(x, t), u_\theta(x, t)) \in \mathbb{R}^2$.

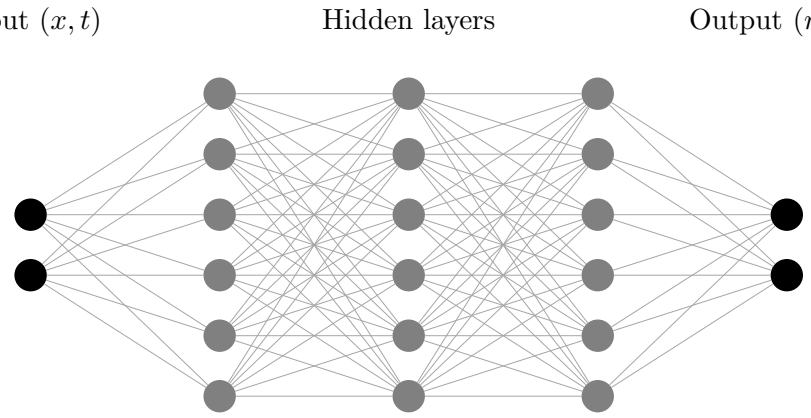


Figure 2.2 – Example of MLP architecture of input (x, t), with $L_{\text{MLP}} = 3$ hidden layers and $N_{\text{MLP}} = 6$ neurons per layer.

Source: The author (2026).

2.3.2 Gradient Flow

To understand how an MLP minimizes its data-fitting loss (2.28), consider gradient descent with learning rate $\mu > 0$. Starting from an initialization $\theta^{(0)}$, each iteration updates the parameters by

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_\theta \mathcal{L}(\theta^{(k)}), \qquad k = 0, 1, 2, \ldots, \tag{2.32}$$

where $\theta^{(k)}$ denotes the parameter vector after k steps. Rearranging (2.32) and dividing by the step size μ isolates the per-step increment as a difference quotient:

$$\frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_\theta \mathcal{L}(\theta^{(k)}). \tag{2.33}$$

To pass from this discrete recursion to a continuous trajectory, assign to each iterate a training-time stamp

$$\tau^{(k)} = k\mu, \qquad \text{so that} \qquad \tau^{(k+1)} - \tau^{(k)} = \mu, \tag{2.34}$$

spacing successive iterates by μ along a time axis τ. We then regard the discrete sequence $\{\theta^{(k)}\}_{k=0}^{\infty}$ as samples of a smooth curve $\theta(\tau)$ evaluated at these instants:

$$\theta^{(k)} = \theta(\tau^{(k)}), \qquad \theta^{(k+1)} = \theta(\tau^{(k+1)}) = \theta(\tau^{(k)} + \mu). \tag{2.35}$$

Substituting (2.35) into the left-hand side of (2.33) recasts it as a forward finite-difference quotient of θ in τ:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_\theta \mathcal{L}\big(\theta(\tau^{(k)})\big). \tag{2.36}$$

As the step size $\mu \to 0$, the spacing in (2.34) shrinks to zero, the discrete stamps $\tau^{(k)}$ fill out a continuous variable τ, and the left-hand side of (2.36) converges to the derivative $d\theta/d\tau$ by definition of the derivative. The recursion thus becomes the gradient flow ODE:

$$\frac{d\theta}{d\tau} = -\nabla_\theta \mathcal{L}(\theta), \qquad \theta(0) = \theta^{(0)}. \tag{2.37}$$

The gradient flow (2.37) is a central object in optimization, and it is a key insight that many standard first-order methods can be recovered as different time-discretization schemes of this same continuous trajectory. The discrete update (2.32) is precisely the forward (explicit) Euler discretization of (2.37) with step size μ, so gradient descent is just one numerical integrator for the flow; applying instead the backward (implicit) Euler scheme recovers the proximal gradient method (PARIKH et al., 2013). Reading optimizers as discretizations of (2.37) thus organizes them into a single family, and it is the continuous flow that we analyze in what follows.

2.3.3 Neural Tangent Kernel

Consider the MLP fit to the dataset $\{(x_i, u_i)\}_{i=1}^{N}$ of Section 2.3.1. Group the pointwise fitting errors into a single error vector:

$$e(\theta) = \begin{bmatrix} u_\theta(x_1) - u_1 \\ \vdots \\ u_\theta(x_N) - u_N \end{bmatrix} \in \mathbb{R}^N. \tag{2.38}$$

Up to the constant factor $1/N$, the loss (2.28) is the squared norm of this vector, which we use in the form

$$\mathcal{L}(\theta) = \frac{1}{2}\,\|e(\theta)\|^2 = \frac{1}{2}\sum_{i=1}^{N} e_i(\theta)^2. \tag{2.39}$$

Define the Jacobian of the error vector with respect to the parameters:

$$J(\theta) = \frac{\partial e}{\partial \theta} \in \mathbb{R}^{N \times N_\theta}, \qquad J_{ij} = \frac{\partial e_i}{\partial \theta_j}, \quad i = 1, \ldots, N,\ j = 1, \ldots, N_\theta, \tag{2.40}$$

where N_θ is the total number of trainable parameters. Each row $J_i = (J_{i1}, \ldots, J_{iN_\theta}) \in \mathbb{R}^{N_\theta}$ is the parameter-gradient of the i-th residual. Since $\mathcal{L} = \frac{1}{2}\sum_i e_i^2$, the j-th component of the loss gradient is:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{i=1}^{N} e_i(\theta)\, \frac{\partial e_i}{\partial \theta_j} = \sum_{i=1}^{N} J_{ij}\, e_i(\theta) = \big(J(\theta)^T e(\theta)\big)_j. \tag{2.41}$$

Collecting all $j = 1, \ldots, N_\theta$ into a column vector:

$$\nabla_\theta \mathcal{L} = J(\theta)^T\, e(\theta) \in \mathbb{R}^{N_\theta}. \tag{2.42}$$

Substituting (2.42) into the gradient flow (2.37):

$$\frac{d\theta}{d\tau} = -J(\theta)^T\, e(\tau). \tag{2.43}$$

To find how the error vector itself evolves, differentiate each component $e_i(\theta(\tau))$ with respect to τ by the chain rule:

$$\frac{de_i}{d\tau} = \sum_{j=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_j}\, \frac{d\theta_j}{d\tau} = J_i(\theta) \cdot \frac{d\theta}{d\tau}. \tag{2.44}$$

Stacking this identity for all $i = 1, \ldots, N$ into a matrix equation:

$$\frac{de}{d\tau} = J(\theta)\, \frac{d\theta}{d\tau}. \tag{2.45}$$

Substituting (2.43):

$$\frac{de}{d\tau} = J(\theta)\big(-J(\theta)^T e(\tau)\big) = -\underbrace{J(\theta)\, J(\theta)^T}_{K(\theta)\, \in\, \mathbb{R}^{N \times N}}\, e(\tau). \tag{2.46}$$

The matrix $K(\theta) = J(\theta)J(\theta)^T$ is the empirical Neural Tangent Kernel (NTK) (JACOT et al., 2018). It is symmetric positive semi-definite by construction: for any $v \in \mathbb{R}^N$,

$$v^T K(\theta)\, v = v^T J(\theta)\, J(\theta)^T v = \big\| J(\theta)^T v \big\|^2 \geq 0. \tag{2.47}$$

The (i, j) entry of $K(\theta)$ follows directly from expanding the matrix product $J(\theta)J(\theta)^T$:

$$K_{ij}(\theta) = \sum_{l=1}^{N_\theta} J_{il}(\theta)\, J_{jl}(\theta) = \sum_{l=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_l}(\theta)\, \frac{\partial e_j}{\partial \theta_l}(\theta) = \nabla_\theta e_i(\theta)^T\, \nabla_\theta e_j(\theta). \tag{2.48}$$

This is the inner product in $\mathbb{R}^{N_\theta}$ of the parameter gradients of the network output at the i-th and j-th training points, where $e_i = u_\theta(x_i) - u_i$. Hence $K(\theta) \in \mathbb{R}^{N\times N}$ is the Gram matrix of the network's sensitivities across the whole dataset, and equation (2.46) shows that it alone governs how the fitting error evolves under gradient flow.

2.3.4 Spectral Bias

The spectral bias (or frequency principle) is the empirical observation that neural networks trained by gradient-based methods reduce the low-frequency components of the error faster than the high-frequency ones (RAHAMAN et al., 2019). This section derives that phenomenon directly from the NTK dynamics established above. When the training points lie on a regular grid and the target function is periodic, the error admits a discrete Fourier representation, and one can ask how its frequency content is approximated along the training iterations. We first establish the decay in the eigenbasis of the NTK and then, by transforming to Fourier space, show that it is precisely the low frequencies that are resolved first.

For a network with a sufficiently large number of parameters, the Jacobian $J(\theta)$ changes negligibly during training (JACOT et al., 2018; WANG; YU, et al., 2022). Writing $J_0 = J(\theta(0))$ and $K_0 = J_0 J_0^T$, the approximation $K(\theta(\tau)) \approx K_0$ turns equation (2.46) into a linear ODE with constant coefficients:

$$\frac{de}{d\tau} = -K_0\, e(\tau), \qquad e(0) = e_0. \tag{2.49}$$

Written component-wise, the i-th row of this system is

$$\frac{de_i}{d\tau} = -\sum_{j=1}^{N} K_0^{ij}\, e_j(\tau), \qquad i = 1, \ldots, N, \tag{2.50}$$

where $K_0^{ij} = \nabla_{\theta_0} e_i^T\, \nabla_{\theta_0} e_j$ is the inner product, at initialization, of the parameter gradients of the network output at training points x_i and x_j. The rate of change of the error at point i is therefore a weighted sum of all current errors, with weights given by these pairwise kernel values. This coupling is what makes the geometry of the sample grid and the architecture of the network jointly determine the convergence of every component.

Since $K_0 \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite, the spectral theorem

guarantees a decomposition

$$K_0 = V\,\Lambda\,V^T, \tag{2.51}$$

where $V = [v_1 \mid \cdots \mid v_N]$ is an orthogonal matrix ($V^TV = I$) and

$$\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_N), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0. \tag{2.52}$$

Introduce the change of coordinates

$$c(\tau) = V^T e(\tau), \tag{2.53}$$

so that, since V is orthogonal, $e(\tau) = Vc(\tau)$. Differentiating (2.53) with respect to τ and substituting the ODE (2.49):

$$\frac{dc}{d\tau} = V^T \frac{de}{d\tau} = -V^T K_0\, e(\tau). \tag{2.54}$$

Inserting $K_0 = V\Lambda V^T$ and $e(\tau) = Vc(\tau)$:

$$\frac{dc}{d\tau} = -V^T\,(V\Lambda V^T)\,(Vc(\tau)) = -(V^TV)\,\Lambda\,(V^TV)\,c(\tau) = -\Lambda\,c(\tau), \tag{2.55}$$

using $V^TV = I$ at each step. Since Λ is diagonal, this decouples into N independent scalar equations:

$$\frac{dc_k}{d\tau} = -\lambda_k\,c_k(\tau), \qquad k = 1, \ldots, N. \tag{2.56}$$

Each equation in (2.56) is a first-order linear ODE with constant coefficient (STROGATZ, 2018). Separating the variables and integrating both sides from the initial state at $\tau = 0$ up to time τ, with s and r as integration variables for the amplitude and the time,

$$\int_{c_k(0)}^{c_k(\tau)} \frac{ds}{s} = -\lambda_k \int_0^{\tau} dr \qquad \Longrightarrow \qquad \ln\frac{|c_k(\tau)|}{|c_k(0)|} = -\lambda_k\,\tau. \tag{2.57}$$

Exponentiating both sides yields the solution

$$c_k(\tau) = c_k(0)\,e^{-\lambda_k\tau}, \tag{2.58}$$

where $c_k(0) = v_k^T e_0$ is the projection of the initial error e_0 onto the k-th eigenvector of K_0. For $\lambda_k = 0$ the equation gives $c_k(\tau) = c_k(0)$ for all τ: that eigendirection never decays.

Returning to the original coordinates via $e(\tau) = Vc(\tau)$:

$$e(\tau) = \sum_{k=1}^{N} \left(v_k^T e_0\right) e^{-\lambda_k\tau}\, v_k. \tag{2.59}$$

The error decomposes into N independent eigendirections. Each eigenvector v_k carries initial amplitude $v_k^T e_0$ and decays exponentially at rate λ_k: eigenvectors with large λ_k are suppressed quickly, while those with small λ_k persist throughout training. This per-eigendirection decay, each at the rate set by its eigenvalue, is the rigorous statement of spectral bias established by Cao et al. (2021); the same spectral ordering governs generalization as the training set grows (BORDELON et al., 2020).

Equation (2.59) is the continuous flow; gradient descent takes finite steps, and its error evolution inherits the same eigenstructure in discrete form. Linearizing the update (2.32) about the frozen Jacobian J_0, a single step moves the parameters by $\theta^{(n+1)} - \theta^{(n)} = -\mu\, J_0^T e^{(n)}$, and to first order the error responds as $e^{(n+1)} - e^{(n)} = J_0\,(\theta^{(n+1)} - \theta^{(n)})$. Combining the two,

$$e^{(n+1)} = e^{(n)} - \mu\, J_0 J_0^T e^{(n)} = (I - \mu K_0)\, e^{(n)}. \tag{2.60}$$

Iterating from $e^{(0)} = e_0$ and diagonalizing with $K_0 = V \Lambda V^T$,

$$e^{(n)} = (I - \mu K_0)^n e_0 = \sum_{k=1}^{N} \big(v_k^T e_0\big)\,(1 - \mu\lambda_k)^n\, v_k. \tag{2.61}$$

This is the forward-Euler counterpart of (2.59): each eigendirection is multiplied by $(1 - \mu\lambda_k)^n$ rather than $e^{-\lambda_k \tau}$, and the two coincide as $\mu \to 0$ with $\tau = n\mu$, since $(1 - \mu\lambda_k)^n \to e^{-\lambda_k \tau}$. The discrete mode contracts only when $|1 - \mu\lambda_k| < 1$, that is $\mu < 2/\lambda_k$, so the largest eigenvalue caps the stable learning rate at $\mu < 2/\lambda_{\max}$.

From (2.58), the amplitude of the k-th eigenvector at time τ satisfies $|c_k(\tau)| = |v_k^T e_0|\, e^{-\lambda_k \tau}$. To reduce this amplitude below a target precision $\varepsilon > 0$:

$$|v_k^T e_0|\, e^{-\lambda_k \tau} < \varepsilon \implies e^{-\lambda_k \tau} < \frac{\varepsilon}{|v_k^T e_0|} \implies -\lambda_k\, \tau < \ln \frac{\varepsilon}{|v_k^T e_0|} \implies \tau > \frac{1}{\lambda_k} \ln \frac{|v_k^T e_0|}{\varepsilon}. \tag{2.62}$$

The minimum training time to resolve the k-th eigenvector to precision ε is therefore

$$\tau_k = \frac{1}{\lambda_k} \ln \frac{|v_k^T e_0|}{\varepsilon}. \tag{2.63}$$

Since $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0$, equation (2.63) gives $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_N$: higher-indexed eigenvectors require more training time. Arora et al. (2019) reached similar conclusions from a discrete-time analysis of gradient descent, showing that the convergence speed of each eigenvector is governed by its eigenvalue and by the projection of the target onto that eigenvector.

The decomposition (2.59) is expressed in the eigenbasis $\{v_k\}_{k=1}^N$ of the NTK, which

need not coincide with any physically meaningful basis. When the N training points lie on a regular grid and the target is periodic, however, the natural basis is the Fourier one, and recasting the dynamics there shows that the decay is genuinely organized by frequency. Let $F \in \mathbb{C}^{N\times N}$ be the Fourier matrix, that is, the discrete Fourier transform (DFT) matrix with entries

$$F_{jl} = \frac{1}{\sqrt{N}}\,\omega^{jl}, \qquad \omega = e^{-2\pi \mathrm{i}/N}, \qquad j,l = 0,\dots,N-1. \tag{2.64}$$

With this normalization F is unitary, $F^*F = I$, so that $F^{-1} = F^*$, where F^* is the conjugate transpose (GOLUB et al., 2013; STRANG, 2006). Applying F to the error vector yields its spectrum, with inverse transform F^{-1}:

$$\hat{e}(\tau) = F\,e(\tau), \qquad e(\tau) = F^{-1}\hat{e}(\tau) = F^*\hat{e}(\tau). \tag{2.65}$$

In particular $\hat{e}_0 = Fe_0$ is the spectrum of the initial error, the difference between the spectra of the initial network output and the target. Since F is constant in τ, applying it to the linearized dynamics (2.49) carries the error evolution into the frequency domain:

$$\frac{d\hat{e}}{d\tau} = F\,\frac{de}{d\tau} = -FK_0\,e(\tau) = -FK_0F^{-1}\,\hat{e}(\tau). \tag{2.66}$$

Inserting the eigendecomposition $K_0 = V\Lambda V^T$ from (2.51),

$$FK_0F^{-1} = FV\,\Lambda\,V^TF^{-1} = (FV)\,\Lambda\,(FV)^* = (FV)\,\Lambda\,(FV)^{-1}, \tag{2.67}$$

where $V^TF^{-1} = V^*F^* = (FV)^*$ uses that V is real orthogonal, and $(FV)^* = (FV)^{-1}$ uses that a product of unitary matrices is unitary. The columns of FV, namely the Fourier transforms $\{Fv_k\}_{k=1}^N$ of the NTK eigenvectors, therefore diagonalize the frequency-domain dynamics. Defining the modal amplitudes $\tilde{c}(\tau) = (FV)^{-1}\hat{e}(\tau)$ and repeating the steps (2.53)–(2.58) gives

$$\frac{d\tilde{c}_k}{d\tau} = -\lambda_k\,\tilde{c}_k(\tau) \qquad \Longrightarrow \qquad \tilde{c}_k(\tau) = \tilde{c}_k(0)\,e^{-\lambda_k\tau}. \tag{2.68}$$

These are the same amplitudes as before: $\tilde{c} = (FV)^{-1}Fe = V^{-1}e = V^Te = c$, the projection of the error onto the NTK eigenvectors, now read in Fourier space. Transforming back through (2.65) yields the evolution of the error spectrum,

$$\hat{e}(\tau) = \sum_{k=1}^{N}\big[(Fv_k)^*\hat{e}_0\big]\,e^{-\lambda_k\tau}\,(Fv_k). \tag{2.69}$$

Equation (2.69) is the spectral-bias statement in the literal sense of the frequency spectrum: the error spectrum is a superposition of the transformed eigenvectors Fv_k, each damped at its own rate λ_k. The frequencies resolved first are those carried by the eigenvectors with the largest eigenvalues; empirically these eigenvectors are smooth and concentrated in the low Fourier modes (RAHAMAN et al., 2019; XU et al., 2020), so the low-frequency content of the error decays first while the high-frequency content persists, exactly the behaviour observed in practice.

To close, note that when the network is initialized at small scale its output u_{θ_0} is negligible relative to the target, a common simplification in spectral-bias analyses. The initial error is then the target with opposite sign, and so is its spectrum:

$$\hat{e}_0 = F\big(u_{\theta_0} - u\big) = \hat{u}_{\theta_0} - \hat{u} \approx -\,\hat{u} = -\begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(N-1) \end{bmatrix}. \tag{2.70}$$

Substituting (2.70) into (2.69) casts the dynamics as a reconstruction of the target frequency by frequency: each Fourier mode of u is acquired at a rate set by the NTK eigenvalues, the smooth low-frequency content emerging first and the fine high-frequency detail last. This is exactly the regime anticipated at the start of the section — a periodic target on a regular grid — in which the quality of the approximation can be tracked mode by mode along the training iterations.

This last observation can be made quantitative. Taking the modulus of the modal amplitude (2.68) isolates the magnitude of the k-th component along training,

$$|\tilde{c}_k(\tau)| = |\tilde{c}_k(0)|\, e^{-\lambda_k \tau}, \qquad |\tilde{c}_k(0)| = |(Fv_k)^* \hat{e}_0| \approx |(Fv_k)^* \hat{u}|, \tag{2.71}$$

where the last estimate uses the small-scale initialization (2.70), so that $|(Fv_k)^*\hat{u}|$ measures how strongly the target overlaps with the k-th eigen-mode. Training, however, advances in discrete steps: by the construction of the gradient flow (2.34), the continuous time and the iteration count n are related by $\tau = n\mu$, with μ the learning rate. Substituting $\tau = n\mu$ into (2.71) and requiring the amplitude to fall below a target precision $\varepsilon > 0$,

$$|(Fv_k)^*\hat{u}|\, e^{-\lambda_k n\mu} < \varepsilon \implies n > \frac{1}{\mu\,\lambda_k}\, \ln \frac{|(Fv_k)^*\hat{u}|}{\varepsilon}, \tag{2.72}$$

gives an estimate of the number of gradient-descent iterations needed to resolve the k-th frequency:

$$n_k = \left\lceil \frac{1}{\mu\,\lambda_k}\, \ln \frac{|(Fv_k)^*\hat{u}|}{\varepsilon} \right\rceil. \tag{2.73}$$

2.3.5 Physics-Informed Neural Networks

Introduced by Raissi et al. (2017), Physics-Informed Neural Networks (PINNs) insert the PDE directly into the optimization process, forcing the neural network to satisfy the model equations and approximating the PDE solution.

Consider the general PDE Initial Value Problem:

$$\begin{cases} \mathcal{D}[u](x,t) = 0, \quad \forall (x,t) \in \Omega \times (0,T], \\ \quad u(x,0) = u_0(x), \quad x \in \Omega, \end{cases} \tag{2.74}$$

where $\mathcal{D}$ is a differential operator on u, $\Omega \times (0,T]$ is the spatio-temporal domain, and u_0 is the initial condition.

The PDE residual loss penalizes violation of the governing equation at collocation points $\{(x_i,t_i)\}_{i=1}^{N_\mathcal{D}} \subset \Omega \times (0,T]$:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_\mathcal{D}} \sum_{i=1}^{N_\mathcal{D}} \left\| \mathcal{D}[u_\theta](x_i,t_i) \right\|^2 . \tag{2.75}$$

The initial condition is enforced at collocation points $\{x_i\}_{i=1}^{N_0} \subset \Omega$:

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left\| u_\theta(x_i,0) - u_0(x_i) \right\|^2 . \tag{2.76}$$

The combined PINN loss is:

$$\mathcal{L}_{\text{PINN}}(\theta) = \mathcal{L}_{\text{pde}}(\theta) + \mathcal{L}_{\text{ic}}(\theta). \tag{2.77}$$

Due to the compositional structure of MLPs, PINN implementations evaluate PDE residuals via Automatic Differentiation (AD) (LINNAINMAA, 1970; PASZKE et al., 2019), using ADAM (KINGMA et al., 2014) or L-BFGS (NOCEDAL, 1980) as the optimizer.

Application to the Boussinesq Dataset

For the Boussinesq system, a separate PINN is trained for each sample $(\alpha_i, \beta_i) \in \mathcal{S}$ (2.27). The differential operator $\mathcal{D}$ in the generic problem (2.74) is instantiated as the pair of Boussinesq residuals with fixed parameters (α_i, β_i):

$$\mathcal{D}_1[u_\theta] = (\eta_\theta)_t + (u_\theta)_x + \alpha_i \left((\eta_\theta \, u_\theta)_x \right), \tag{2.78}$$

$$\mathcal{D}_2[u_\theta] = (u_\theta)_t - \tfrac{\beta_i}{3} (u_\theta)_{xxt} + (\eta_\theta)_x + \alpha_i \, (u_\theta (u_\theta)_x). \tag{2.79}$$

The combined loss (2.77) for sample i then reads:

$$\begin{aligned}\mathcal{L}^i_{\mathrm{PINN}}(\theta) = \frac{1}{N_{\mathcal{D}}}\sum_{l=1}^{N_{\mathcal{D}}}\Big[\mathcal{D}_1[u_\theta](x_l,t_l)^2 + \mathcal{D}_2[u_\theta](x_l,t_l)^2\Big] \\ + \frac{1}{N_0}\sum_{l=1}^{N_0}\Big[(\eta_\theta(x_l,0)-\eta_0(x_l))^2 + (u_\theta(x_l,0)-u_0(x_l))^2\Big],\end{aligned} \tag{2.80}$$

where η_0 and u_0 are given by the shared initial condition (2.1). Because the parameters (α_i, β_i) enter the residuals explicitly, each PINN encodes knowledge of a single sample and must be retrained for every new parameter pair.

2.3.6 Spectral Bias in Physics-Informed Neural Networks

The Neural Tangent Kernel analysis of Sections 2.3.3–2.3.4 was carried out for an MLP fitting data on a uniform grid, the setting in which the kernel is approximately circulant and its eigenvectors are genuine Fourier modes. A PINN does not fit data directly: its loss (2.77) combines an initial-condition term with a PDE-residual term. The same machinery nevertheless applies once both constraints are stacked into a single error vector, now indexed over the N_r residual points in place of the N data points of the MLP,

$$e(\theta) = \begin{bmatrix} u_\theta(x_1,0)-u_0(x_1) \\ \vdots \\ u_\theta(x_{N_0},0)-u_0(x_{N_0}) \\ \mathcal{D}[u_\theta](x_1,t_1) \\ \vdots \\ \mathcal{D}[u_\theta](x_{N_{\mathcal{D}}},t_{N_{\mathcal{D}}}) \end{bmatrix} \in \mathbb{R}^{N_r}, \qquad N_r = N_0 + N_{\mathcal{D}}, \tag{2.81}$$

so that, up to normalization, $\mathcal{L}_{\mathrm{PINN}}$ reduces to the same quadratic form $\frac{1}{2}\|e\|^2$ analyzed for the MLP. With this definition the gradient-flow derivation leading to (2.46) carries over verbatim, with the Jacobian now also containing the parameter gradients of the PDE residuals, and the error still obeys the linear dynamics $\frac{de}{d\tau} = -K_0\, e$, with convergence of each eigenvector governed by its eigenvalue exactly as in (2.63) (WANG; YU, et al., 2022).

This prediction is borne out empirically. Spectral bias is among the most consistently reported obstacles in PINN training: networks fit smooth, low-frequency solutions readily but stall on sharp gradients and high-frequency content (RAHAMAN et al., 2019; XU et al., 2020; WANG; TENG, et al., 2021), and the same pathology underlies several documented PINN failure modes (KRISHNAPRIYAN et al., 2021). Wang, Yu, et al. (2022) traced the effect directly to the NTK, showing that its spectrum is concentrated at

low eigenvalues, so that high-frequency PDE residuals are the last to be minimized; for the Boussinesq system specifically, Wang et al. (2024) report the same difficulty in resolving the finer wave structure. A range of remedies target precisely this imbalance, including adaptive loss weighting (WANG; TENG, et al., 2021), Fourier feature embeddings, and domain decomposition. Understanding the spectral bias is therefore essential background for the neural-operator methods studied in the remainder of this work, whose ability to capture the high-frequency content of the Boussinesq solutions is examined directly in the numerical results of Chapter 4.

2.4 Neural Operators

Neural operators generalize neural networks from learning finite-dimensional mappings to learning operators between infinite-dimensional function spaces. This section introduces the general framework, then the Fourier Neural Operator (FNO) as the principal architecture, and finally the Physics-Informed Neural Operator (PINO) as the physics-constrained extension.

2.4.1 General Definition

Let $D \subset \mathbb{R}^d$ be a bounded open domain and let $\mathcal{A}(D;\mathbb{R}^{d_a})$ and $\mathcal{U}(D;\mathbb{R}^{d_u})$ be Banach spaces of functions (KREYSZIG, 1978). The target operator is a (potentially nonlinear) map

$$G^\dagger : \mathcal{A} \to \mathcal{U}, \qquad a \mapsto u = G^\dagger(a), \tag{2.82}$$

where both the input a and the output u are functions defined on D.

In this work, $G^\dagger$ is the solution operator of the Boussinesq system: given a parameter encoding (c, u_0) containing the PDE coefficients (α, β) and the initial condition u_0, the operator returns the full space-time solution (η, u).

A neural operator G_θ approximates $G^\dagger$ by composing linear integral operators with pointwise nonlinearities. Each linear layer applies an operator of the form:

$$(\mathcal{K}v)(x) = \int_D \kappa(x,y)\, v(y)\, dy, \tag{2.83}$$

where $\kappa : D \times D \to \mathbb{R}^{d_u \times d_u}$ is a learned kernel. The full neural operator architecture stacks these integral layers into the composition:

$$G_\theta = Q \circ \sigma\Big(W_L + \mathcal{K}_L\Big) \circ \cdots \circ \sigma\Big(W_1 + \mathcal{K}_1\Big) \circ P, \tag{2.84}$$

where:

- $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_c}$ is the lifting operator, mapping the input $a(x)$ pointwise to a higher-dimensional channel representation with $d_c \gg d_a$;
- $\sigma\Big(W_\ell + \mathcal{K}_\ell\Big)$, for $\ell = 1, \ldots, L$, are the operator layers, each combining a local linear map W_ℓ with a nonlocal integral operator $\mathcal{K}_\ell$, followed by a pointwise activation σ;
- $Q : \mathbb{R}^{d_c} \to \mathbb{R}^{d_u}$ is the projection operator mapping the final channel representation back to the output dimension.

Different choices of kernel parametrization for each $\mathcal{K}_\ell$ give rise to different neural operator architectures.

2.4.2 Fourier Neural Operators

Presented by Li et al. (2020), the Fourier Neural Operator (FNO) is the neural operator (2.84) in which each $\mathcal{K}_\ell$ uses a shift-invariant kernel: $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. This assumption turns each integral operator into a convolution, which by the convolution theorem can be evaluated efficiently in the frequency domain.

Each spectral layer updates the channel representation via:

$$v_{\ell+1}(x) = \sigma\Big((W_\ell\, v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\Big). \tag{2.85}$$

The operator $\mathcal{K}_\ell$ acts via convolution with the shift-invariant kernel κ_ℓ:

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y)\, v_\ell(y)\, dy. \tag{2.86}$$

By the convolution theorem, this equals:

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell)\Big)(x). \tag{2.87}$$

To see how this is parametrized, recall that for a periodic function f on $[0, 2\pi]$:

$$f(x) = \sum_{k\in\mathbb{Z}} \hat{f}(k)\, e^{ikx}, \qquad \hat{f}(k) = \frac{1}{2\pi}\int_0^{2\pi} f(x) e^{-ikx}\, dx. \tag{2.88}$$

The convolution $(\kappa * v)$ simplifies to:

$$(\kappa * v)(x) = \int_0^{2\pi} \kappa(x - y)\, v(y)\, dy = \sum_{k\in\mathbb{Z}} \big(2\pi\, \hat{\kappa}(k)\, \hat{v}(k)\big)\, e^{ikx}. \tag{2.89}$$

Instead of learning κ_ℓ as a function in physical space, the FNO directly parametrizes its Fourier coefficients as learnable complex-valued matrices $R_{\ell,k} \approx 2\pi\, \hat{\kappa}_\ell(k)$. Only the

$|k| \leq k_{\max}$ low-frequency modes are retained, with all others set to zero; $k_{\max}$ is a hyperparameter.

Training Objective

The FNO is trained as a pure supervised learning problem. Given the dataset $\mathcal{D}$ (2.27) of parameter-solution pairs, the training objective minimizes the mean squared error between the predicted and reference space-time fields:

$$\mathcal{L}_{\text{FNO}}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n)-\eta_i(x_j,t_n))^2+(u_\theta^i(x_j,t_n)-u_i(x_j,t_n))^2\Big], \tag{2.90}$$

where $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ is the network prediction for sample i.

Application to Non-Time-Periodic Problems

When the solution is also assumed to be periodic in time with period T (so that $D = [0, 2\pi] \times [0, T]$), shift-invariance extends to the time direction as well: $\kappa_\ell(x, y, t, \tau) = \kappa_\ell(x - y,\, t - \tau)$. The convolution then generalizes to:

$$(\mathcal{K}_\ell v_\ell)(x,t) = \int_D \kappa_\ell(x-y,\, t-\tau)\, v_\ell(y,\tau)\, dy\, d\tau = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell)\cdot\mathcal{F}(v_\ell)\Big)(x,t), \tag{2.91}$$

and the FNO parametrizes the two-dimensional Fourier modes (k_x, k_t) with $|k_x| \leq k_{x,\max}$ and $|k_t| \leq k_{t,\max}$, both hyperparameters.

For problems that are not time-periodic, however, applying the Fourier transform along the temporal axis implicitly extends the signal periodically, potentially introducing a jump discontinuity at the temporal boundary. This can trigger the Gibbs phenomenon (GIBBS, 1899), producing oscillatory artifacts near the boundary that persist regardless of how many temporal modes are retained. Standard signal-processing techniques such as zero-padding and window functions (e.g. Hann, Hamming windows) can partially mitigate these effects, at the cost of altering the temporal structure of the signal. Li et al. (2020) acknowledge this limitation and provide an alternative formulation in which the Fourier layers operate only along the spatial axis while the model advances forward in time, avoiding the periodicity assumption in the temporal direction entirely. In this work we deliberately retain the two-dimensional space-time formulation, in which the Fourier layers act on both the spatial and the temporal axes, so that a single operator emits the entire trajectory in one forward pass. This keeps the architecture simple and uniform across space and time, but, as just noted, it reintroduces the temporal-periodicity assumption for a Boussinesq solution that is not time-periodic, and therefore admits a Gibbs artifact

at the temporal boundary $t = T$. Rather than suppress this artifact with windowing, we retain the plain formulation and examine the boundary behaviour directly in Chapter 4, where it turns out to distinguish the purely data-driven operator from its physics-informed variant.

2.4.3 Physics-Informed Neural Operators

Analogously to how PINNs add a physics-informed residual loss to the MLP training objective, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) incorporates PDE constraints into the FNO training. The FNO backbone remains unchanged; what differs is the loss function, which now combines a data term with residuals evaluated on the predicted space-time fields.

Unlike a PINN, which represents the solution as a single continuous function and differentiates it via automatic differentiation, the FNO outputs (η_θ, u_θ) as a discrete $N_x \times N_t$ array in a single forward pass over the full space-time domain. Evaluating the PDE residuals therefore reduces to numerically differentiating this output array.

Spatial derivatives are computed pseudospectrally using ξ_k (2.18): the DFT of η_θ along the spatial axis at each time t_n yields $\hat{\eta}_\theta(k, t_n)$, from which

$$(\eta_x)_j = \mathcal{F}^{-1}(i\,\xi_k\,\hat{\eta}_\theta(k,t_n))_j\,, \qquad (\eta_{xx})_j = \mathcal{F}^{-1}\big(-\xi_k^2\,\hat{\eta}_\theta(k,t_n)\big)_j\,, \tag{2.92}$$

and identically for u_θ. Time derivatives are computed by finite differences: second-order central in the interior and first-order one-sided at the boundaries,

$$(\eta_\theta)_t^n = \begin{cases} \dfrac{\eta_\theta^{n+1} - \eta_\theta^n}{\Delta t}, & n = 0, \\ \dfrac{\eta_\theta^{n+1} - \eta_\theta^{n-1}}{2\,\Delta t}, & 1 \le n \le N_t - 1, \\ \dfrac{\eta_\theta^n - \eta_\theta^{n-1}}{\Delta t}, & n = N_t, \end{cases} \tag{2.93}$$

and identically for $(u_\theta)_t^n$. The mixed term $(u_\theta)_{xxt}$ is obtained by first computing $(u_\theta)_{xx}$ spectrally and then applying the time-difference stencil to the result. This combination of spectral spatial derivatives and finite-difference temporal derivatives follows the reference implementation of Li et al. (2023a) (LI et al., 2023b).

With all derivatives available, the Boussinesq equations (2.1) define two differential operators acting on the predicted fields:

$$\mathcal{D}_1(\eta_\theta, u_\theta) \coloneqq (\eta_\theta)_t + (u_\theta)_x + \alpha\,(\eta_\theta\,u_\theta)_x, \tag{2.94}$$

$$\mathcal{D}_2(\eta_\theta, u_\theta) \coloneqq (u_\theta)_t - \tfrac{\beta}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha\,(u_\theta(u_\theta)_x). \tag{2.95}$$

Here $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ denotes the network output for sample i, and the subscript $_{j,n}$ indicates evaluation at the grid point (x_j, t_n). The PINO loss is a weighted sum of three terms:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}(\theta), \tag{2.96}$$

where:

- $\mathcal{L}_{\text{pde}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[\mathcal{D}_1(\eta_\theta^i, u_\theta^i)_{j,n}^2 + \mathcal{D}_2(\eta_\theta^i, u_\theta^i)_{j,n}^2\Big]$ penalizes violation of the Boussinesq equations at each grid point, with residuals evaluated using the parameters (α_i, β_i) of each sample;
- $\mathcal{L}_{\text{ic}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x}\sum_{j=0}^{N_x-1}\Big[(\eta_\theta(x_j,0) - \eta_0(x_j))^2 + (u_\theta(x_j,0) - u_0(x_j))^2\Big]$ enforces the shared initial condition (2.1);
- $\mathcal{L}_{\text{data}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n) - \eta_i(x_j,t_n))^2 + (u_\theta^i(x_j,t_n) - u_i(x_j,t_n))^2\Big]$ measures deviation from the reference solutions $(\eta_i, u_i) \in \mathcal{D}$ (2.27);
- λ_{pde}, λ_{ic}, $\lambda_{\text{data}} > 0$ are weighting hyperparameters.

The PINO therefore generalizes the FNO by adding physics supervision: it can be trained with fewer labeled samples by relying on the PDE residual for additional guidance.

3 METHODOLOGY

This chapter describes how the experiments of this work were designed, implemented and evaluated. Section 3.1 states the computational environment and the reproducibility conventions shared by every experiment. Section 3.2 details the generation of the reference dataset with the pseudospectral solver of Section 2.2 and its encoding as tensors. Section 3.3 defines the error metrics and the common evaluation protocol against which all methods are compared. Sections 3.4, 3.5 and 3.6 describe, respectively, the PINN, FNO and PINO experiments, including the discretization of each differential operator. Section 3.7 describes the Neural Tangent Kernel diagnostic used to probe spectral bias directly. Table 3.2 collects all hyperparameters in a single place.

3.1 Experimental Setup and Reproducibility

All experiments were implemented from scratch in Python 3.11 using PyTorch 2.12 with CUDA 13.0 support (PASZKE et al., 2019), and were executed on a single NVIDIA GeForce GTX 1660 SUPER graphics card (6 GB of memory). Numerical array manipulation relied on NumPy, while the figures were produced with Matplotlib and Plotly. No pre-trained weights, external datasets or high-level operator-learning libraries were used: the pseudospectral solver, the multilayer perceptron, the Fourier layers and every training loop were coded directly, so that each architecture could be instrumented for the spectral diagnostics reported in Chapter 4. The complete source code — the pseudospectral solver, all three architectures and every plotting routine — is publicly available at `https://github.com/samuelkutz/TCC`.

The three architectures are not third-party implementations but faithful reconstructions of the methods proposed by their original authors: the Fourier Neural Operator follows Li et al. (2020, 2021), the Physics-Informed Neural Operator follows Li et al. (2023a) and its reference code (LI et al., 2023b), and the Physics-Informed Neural Network follows Raissi et al. (2019). Where an implementation choice departs from the reference, the departure is stated explicitly in the corresponding section.

To make the results reproducible, a single global seed (37) was fixed for the Python, NumPy and PyTorch random number generators before every run, and the cuDNN back-

end was set to deterministic mode with autotuning disabled. Unless otherwise noted, the spatial and temporal domains were held fixed at

$$x \in [-L, L], \quad L = 60, \qquad t \in [0, T], \quad T = 30, \tag{3.1}$$

and the shared initial condition of Equation (2.1) was used throughout with unit amplitude $A = 1$, so that $\eta(x, 0) = \operatorname{sech}^2(x)$ and $u(x, 0) = 0$. This places a narrow crest of unit height and unit width at the centre of a domain of half-length 60, so that the two counter-propagating pulses anticipated by the wave-equation limit (2.10)–(2.13) never reach the periodic boundary within the simulated horizon.

Table 3.1 collects the fixed simulation, dataset and evaluation parameters shared by every experiment; the individual sections below restate the relevant entries where they are first used.

Table 3.1 – Fixed simulation, dataset and evaluation parameters shared by every experiment. Unless a section states otherwise, these values hold throughout.

Parameter	**Value**
Domain and initial condition	
Spatial half-length L $(\Omega = [-L, L])$	60
Final time T $(t \in [0, T])$	30
Initial wave amplitude A	1
Initial condition	$\eta(x, 0) = \operatorname{sech}^2(x),\ u(x, 0) = 0$
Discretization (per solve)	
Spatial grid points N_x	128
Time levels N_t (127 RK4 steps)	128
Spatial spacing $\Delta x = 2L/N_x$	0.9375
Temporal spacing $\Delta t = T/(N_t - 1)$	≈ 0.2362
Parameter sampling	
Coupled nonlinearity and dispersion	$\alpha = \beta$
Number of training samples M	20
Sampling grid	linspace(0.1, 4.0, 20)
Evaluation and reproducibility	
Evaluation parameters $\alpha = \beta$	{0.1, 3.21, 4.2}
Evaluation resolutions N	{128, 256, 512}
Spectral evaluation resolution	256
Global random seed	37

Source: The author (2026).

3.2 Reference Dataset Generation

The reference solutions play a dual role in this work: they constitute the supervised training targets for the data-driven regimes, and they serve as the ground truth against

which every method is measured. Both roles are filled by the pseudospectral Runge–Kutta solver of Section 2.2, whose spectral spatial accuracy makes it the natural high-fidelity baseline for a dispersive wave problem (ENGSIG-KARUP et al., 2012; TREFETHEN, 2000).

3.2.1 The One-Parameter Family

As anticipated in Section 2.2.4, the experiments vary only the PDE coefficients while keeping the initial condition fixed. Throughout this work the nonlinearity and dispersion parameters are held equal, $\alpha = \beta$, which collapses the two-parameter Boussinesq family into a single scalar axis of difficulty: for small $\alpha = \beta$ the dynamics are nearly linear and the solution energy is concentrated in a few low-frequency modes, whereas for large $\alpha = \beta$ the coupling between steepening and dispersion produces sharper crests whose energy spreads across a broader band of wavenumbers. The dataset samples this axis on a uniform grid of $M = 20$ values,

$$\alpha_i = \beta_i \in \text{linspace}(0.1,\, 4.0,\, 20), \qquad i = 1, \ldots, 20, \tag{3.2}$$

ranging from the near-linear regime (0.1) to a strongly nonlinear one (4.0).

3.2.2 Discretization and Tensor Encoding

For each parameter value in (3.2), the pseudospectral solver was run on a spatial grid of $N_x = 128$ points and advanced for $N_t = 128$ time levels (that is, 127 Runge–Kutta steps) over $[0, T]$, yielding the grid spacings

$$\Delta x = \frac{2L}{N_x} = \frac{120}{128} = 0.9375, \qquad \Delta t = \frac{T}{N_t - 1} = \frac{30}{127} \approx 0.2362. \tag{3.3}$$

Each solve produces the two space-time fields $\eta_i(x_j, t_n)$ and $u_i(x_j, t_n)$ on the 128×128 grid. These are assembled into input and output tensors in the channel-first convention expected by the two-dimensional operators. The input tensor carries four channels and the output tensor two:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \qquad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t}. \tag{3.4}$$

The four input channels encode, for every grid point, the initial surface elevation $\eta_i(x_j, 0)$, the initial velocity $u_i(x_j, 0)$, and the constant fields α_i and β_i; the initial-condition channels are broadcast (held constant) along the time axis so that the operator receives a full space-time array as input. The two output channels hold the reference fields η_i and u_i. In all

subsequent panels only the surface elevation η is reported, as it carries the wave structure of interest; the velocity field behaves analogously.

3.2.3 Normalization

Because the input channels mix physically distinct quantities (a surface elevation of order one, a velocity field, and coefficients ranging over more than one order of magnitude), all tensors were rescaled to the unit interval by an affine, channel-wise min–max map computed once over the whole dataset,

$$\tilde{v} = \frac{v - v_{\min}}{v_{\max} - v_{\min} + \varepsilon}, \qquad \varepsilon = 10^{-12}, \tag{3.5}$$

with $v_{\min}$ and $v_{\max}$ taken per channel across the sample, spatial and temporal axes. The operators are trained and queried in this normalized space, and their predictions are mapped back to physical units by the inverse of (3.5) before any error metric or physics residual is evaluated. We note a subtlety that is relevant to the interpretation of the physics-only regimes of Sections 3.6: the normalization constants $v_{\min}, v_{\max}$ for the output channels are derived from the reference solutions, so even a model trained without an explicit data-fitting term inherits the output scale of the dataset through Equation (3.5). The "no-data" label should therefore be read as "no supervised residual on the interior fields", not as complete independence from the reference.

3.3 Evaluation Protocol and Error Metrics

All methods are evaluated against freshly computed pseudospectral references and compared through a common battery of metrics, so that differences reflect the architectures rather than the measurement. Three parameter values were chosen to span the difficulty axis,

$$\alpha = \beta \in \{0.1,\ 3.21,\ 4.2\}, \tag{3.6}$$

representing the near-linear, intermediate and strongly nonlinear regimes; the median value 3.21 is used whenever a single representative parameter is required, and is also the value at which the single-parameter PINNs of Section 3.4 are trained. To probe the discretization-invariance claimed for neural operators, each trained operator is additionally queried, without retraining, on grids of resolution

$$N \in \{128,\ 256,\ 512\}, \tag{3.7}$$

that is, at the training resolution and at two and four times finer. The input channels of Section 3.2.2 are simply re-sampled on the finer grid, and the reference is recomputed at

the same resolution.

3.3.1 Metrics

Let η_{true} and η_{pred} denote the reference and predicted surface elevation on a common space-time grid. Three complementary quantities are reported.

Time-resolved relative error.

At each time level t_n, the spatial L^2 discrepancy is normalized by the norm of the reference at that instant,

$$\mathcal{E}(t_n) = \frac{\big\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\big\|_2}{\big\|\eta_{\text{true}}(\cdot, t_n)\big\|_2}. \tag{3.8}$$

Plotting $\mathcal{E}(t_n)$ against time exposes how the error is distributed along the trajectory, and its distribution over all time levels is summarized as a box plot. This is the metric behind the parameter and resolution panels.

Spectral relative error.

Applying the spatial real DFT at a fixed time and comparing amplitudes mode by mode isolates *which* spatial frequencies are misrepresented. Writing $\hat{\eta}(k, t_n)$ for the spatial DFT amplitude at wavenumber index k, the per-mode relative error is

$$\mathcal{E}_{\text{spec}}(k, t_n) = \frac{\big|\,|\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)|\,\big|}{\max\big(|\hat{\eta}_{\text{true}}(k, t_n)|,\, \delta\big)}, \tag{3.9}$$

where a small floor δ, proportional to the peak amplitude, prevents spurious blow-up at modes whose reference amplitude is essentially zero. Its average over the retained modes gives a scalar spectral error per time level; both the mode-resolved curves (at $t = 0$, $t = T/2$ and $t = T$) and the time evolution of the average are reported in the spectral panels.

Frequency-band error during training.

To watch spectral bias unfold along the optimization, the spatial spectrum of the error is split into three contiguous wavenumber bands and each band error is logged every 500 iterations. With N_x the reference resolution, the low, mid and high bands correspond to

$$k < \tfrac{N_x}{4}, \qquad \tfrac{N_x}{4} \le k < \tfrac{N_x}{2}, \qquad k \ge \tfrac{N_x}{2}, \tag{3.10}$$

and the band error is the time-averaged spectral amplitude error of Equation (3.9) restricted to that band. These curves are the raw material of the spectral-bias comparison in Section 4.3. The band error of each operator is tracked on the first dataset sample ($\alpha = \beta = 0.1$), whereas the single-parameter PINNs are tracked on their own training solution at $\alpha = \beta = 3.21$; this difference in reference sample is accounted for in the discussion of Chapter 4.

3.4 Physics-Informed Neural Network Experiment

The PINN, as developed in Section 2.3.5, represents the solution as a single continuous map $(x, t) \mapsto (\eta_\theta, u_\theta)$ and is therefore tied to one parameter pair at a time. Every PINN in this work is a fully connected network with two input units, two output units, four hidden layers of 256 neurons each and hyperbolic-tangent activations, for a total of 198,658 trainable parameters. Optimization uses Adam (KINGMA et al., 2014) with learning rate 10^{-3} for 15,000 iterations.

The composite loss of Equation (2.80) is assembled from three terms. The residual term is evaluated at 5,000 interior collocation points, resampled uniformly at random over $[-L, L] \times [0, T]$ at every iteration; the derivatives entering the Boussinesq operators $\mathcal{D}_1, \mathcal{D}_2$ are obtained exactly by automatic differentiation of the network, without any discretization error. The initial-condition term is enforced at 500 equispaced points at $t = 0$. The optional data term, when present, is a supervised mean-squared error against a subsample of the pseudospectral reference solution.

Two regimes are studied, distinguished only by the weight of the data term:

- **PINN (no data)**: purely physics-informed, with data weight 0; the network sees only the equation and the initial condition.
- **PINN (with data)**: hybrid, with data weight 1; the network additionally fits the reference solution at the training parameter.

Both regimes are trained at the median parameter $\alpha = \beta = 3.21$, the intermediate difficulty of (3.6), which is deliberately chosen in the nonlinear range where spectral bias is expected to be most visible (WANG et al., 2024; ZHENG et al., 2023).

3.5 Fourier Neural Operator Experiment

The FNO learns the parameter-to-solution map for the entire family (3.2) in a single training. Its architecture instantiates the general operator (2.84) with the spectral kernel of Section 2.4.2: a pointwise lifting maps the six input features at each grid point (the four channels of (3.4) together with the two normalized grid coordinates) into a

channel width of 32; four Fourier layers each combine a spectral convolution, truncated to 16 modes per axis, with a local 1×1 convolution and a GELU activation; and a pointwise projection returns the two output channels. This yields 1,057,698 trainable parameters. Training minimizes the supervised mean-squared error (2.90) on the normalized fields, using Adam with learning rate 10^{-3} for 15,000 iterations. Because the dataset comprises only $M = 20$ samples, each iteration is effectively a full-batch gradient step.

On the treatment of time.

A point of implementation must be made explicit, as it bears directly on the results of Chapter 4. Each Fourier layer applies a two-dimensional real DFT over the last two axes of the feature array, which are the spatial and *temporal* axes. The operator therefore treats time as a second periodic spectral dimension rather than as a marching direction, so the full space-time solution is produced in a single forward pass. This choice keeps the architecture simple and lets a single operator emit the entire trajectory at once, but it reintroduces the periodicity assumption discussed in Section 2.4.2: because the Boussinesq solution is not periodic in time, the implicit wrap-around at the temporal boundary $t = T$ excites the Gibbs phenomenon (GIBBS, 1899). The resulting boundary artifact is not a defect to be hidden but a measurable behaviour that we analyze directly, and that turns out to distinguish the FNO from its physics-informed counterpart in Section 4.5.

3.6 Physics-Informed Neural Operator Experiment

The PINO reuses the FNO backbone of Section 3.5 without modification and differs only in its training objective, which augments the data loss with the Boussinesq residual (2.96). Evaluating that residual requires differentiating the predicted space-time array, and, following the reference implementation (LI et al., 2023b), the two directions are treated differently.

Spatial derivatives are computed pseudospectrally, using the wavenumbers ξ_k of Equation (2.18): the spatial DFT of the predicted field is multiplied by $i\xi_k$ for the first derivative and by $-\xi_k^2$ for the second, then transformed back. Temporal derivatives are computed by finite differences on the grid: second-order central differences in the interior and first-order one-sided differences at the two temporal boundaries, exactly as written in Equation (2.94)–(2.95). The mixed term u_{xxt} is obtained by first taking the spatial second derivative spectrally and then applying the temporal stencil. Crucially, the residual is assembled in *physical* units: the normalized network output and the normalized coefficients are mapped back through the inverse of (3.5) before the derivatives are formed, so that the constraint enforces the true Boussinesq balance rather than a rescaled surrogate.

The loss is the weighted sum

$$\mathcal{L}_{\text{PINO}} = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}} + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}} + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}, \tag{3.11}$$

with the interior residual, the initial-condition term and the supervised term of Equation (2.96). As with the PINN, two regimes are compared, differing only in the data weight:

- **PINO (with data)**: $\lambda_{\text{pde}} = \lambda_{\text{ic}} = \lambda_{\text{data}} = 1$; physics and data act together.
- **PINO (no data)**: $\lambda_{\text{data}} = 0$; the operator is guided only by the equation and the initial condition, subject to the normalization caveat of Section 3.2.3.

Optimization again uses Adam with learning rate 10^{-3} for 15,000 iterations. The remaining hyperparameters coincide with the FNO, so that any difference in behaviour is attributable to the physics term alone.

3.7 Neural Tangent Kernel Diagnostic

To connect the observed spectral bias to the theory of Section 2.3.4, a separate diagnostic tracks the Neural Tangent Kernel of the PINN residual directly. Three networks of increasing width, $N_{\text{MLP}} \in \{8, 128, 512\}$ with four hidden layers, are trained at $\alpha = \beta = 3.21$ on the physics-only objective by stochastic gradient descent with a small learning rate (10^{-5}), the regime in which the frozen-kernel approximation (2.49) is expected to hold.

The empirical kernel $K = JJ^{\mathsf{T}}$ is assembled explicitly at a fixed set of 40 probe points by taking, through automatic differentiation, the parameter gradient of each Boussinesq residual and forming the Gram matrix of Equation (2.48). Three quantities are logged every 500 iterations: the relative drift of the parameters $\|\theta - \theta_0\|/\|\theta_0\|$, the relative drift of the kernel $\|K - K_0\|/\|K_0\|$, and the eigenvalue spectrum of K at initialization and at the end of training. The first two test whether the kernel is approximately constant, as the lazy-training analysis assumes; the third exposes the eigenvalue disparity that Equation (2.63) identifies as the mechanism of spectral bias.

3.8 Summary of Hyperparameters

Table 3.2 collects the configuration of every experiment. The three architectures are deliberately trained under a common budget (15,000 iterations, learning rate 10^{-3} for the Adam-optimized models) so that the comparison of Chapter 4 isolates the effect of the architecture and of the training regime rather than of the optimization effort.

Table 3.2 – Configuration of the four training experiments and the NTK diagnostic. The FNO and PINO share the same operator backbone and therefore the same parameter count; the PINNs are single-parameter models. All Adam-trained models use 15,000 iterations.

Setting	**PINN**	**FNO**	**PINO**	**NTK probe**
Trainable parameters	198,658	1,057,698	1,057,698	8/128/512 width
Hidden layers	4	4 Fourier	4 Fourier	4
Width / channels	256	32	32	$8, 128, 512$
Fourier modes (per axis)	—	16	16	—
Optimizer	Adam	Adam	Adam	SGD
Learning rate	10^{-3}	10^{-3}	10^{-3}	10^{-5}
Iterations	15,000	15,000	15,000	15,000
Parameters trained	single (3.21)	all 20	all 20	single (3.21)
Collocation / batch	5,000 pts	full batch	full batch	40 probe pts
Data regimes	with / without	supervised	with / without	physics only

Source: The author (2026).

4 NUMERICAL RESULTS

This chapter reports and interprets the experiments described in Chapter 3. The presentation is organized around a single question: how faithfully does each architecture, in each training regime, reproduce the pseudospectral reference, and how does that fidelity depend on the wavenumber, on the elapsed time, on the nonlinearity, and on the discretization? Section 4.1 reports the training cost. Section 4.2 examines the Neural Tangent Kernel diagnostic that grounds the discussion of spectral bias. Section 4.3 follows the frequency content of the error along training, and Section 4.4 inspects the final spectral fidelity at the intermediate parameter. Section 4.5 isolates the temporal-boundary artifact that separates the data-driven operator from its physics-informed variant, Section 4.6 sweeps the nonlinearity axis, and Section 4.7 tests the zero-shot resolution behaviour. Section 4.8 draws the threads together. Throughout, the pseudospectral solver is treated as ground truth, and no architecture is credited with a success it does not demonstrate: every method examined here leaves a clearly identifiable failure mode.

4.1 Training Cost

Table 4.1 summarizes the size, wall-clock training time and final training loss of the five trained models. The two operators are roughly five times larger than the single-parameter PINN and take about a fifth longer to train, but in exchange a single operator covers the entire 20-point parameter family (3.2), whereas each PINN is bound to the single parameter $\alpha = \beta = 3.21$ and would have to be retrained from scratch for any other value. Amortized over the family, the operators are by far the cheaper way to obtain a solution at a new parameter, since inference is a single forward pass.

A caution is worth stating at the outset. The final-loss column tempts a ranking, but the losses are not on a common scale: the PINN with data reaches a larger final loss (1.1×10^{-4}) than the PINN without data (6.4×10^{-6}) simply because the added supervised term contributes its own residual to the total, not because it is a worse solution — indeed the following sections show the opposite. Training loss is a poor proxy for accuracy here, and all comparisons below rest on the held-out relative-error metrics of Section 3.3.1.

Table 4.1 – Model size, training time on the GTX 1660 SUPER, and final training loss. The final-loss column is *not* comparable across rows: each objective is a different combination of data, residual and initial-condition terms, so its magnitude reflects the composition of the loss rather than solution accuracy. Solution accuracy is measured instead by the relative-error metrics of the following sections.

Model	Regime	Parameters	Train time (s)	Final loss
FNO	supervised	1,057,698	1121.6	5.9×10^{-6}
PINO	with data	1,057,698	1175.5	2.2×10^{-5}
PINO	no data	1,057,698	1176.2	7.2×10^{-6}
PINN	with data	198,658	861.8	1.1×10^{-4}
PINN	no data	198,658	842.2	6.4×10^{-6}

Source: The author (2026).

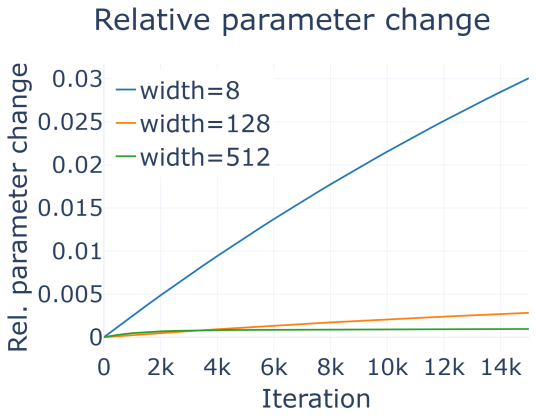
4.2 The Mechanism of Spectral Bias: NTK Diagnostic

Before comparing architectures, we verify that the spectral-bias mechanism derived in Section 2.3.4 is actually operative for the Boussinesq PINN. Figure 4.1 reports the Neural Tangent Kernel diagnostic of Section 3.7 for three network widths.

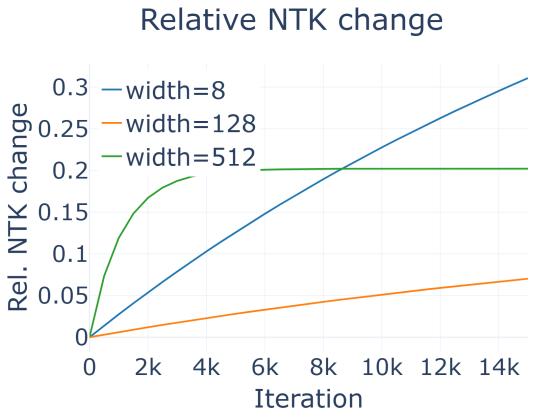
Two predictions of the theory are confirmed. First, the frozen-kernel assumption underlying Equation (2.49) becomes more accurate as the width grows: the widest network (512) drifts by less than 0.1% in its parameters and by under 10% in its kernel over the whole run, whereas the narrowest (8) drifts by several percent in parameters and by more than 40% in its kernel. Wide networks sit closer to the lazy-training regime in which the analysis is exact, and the ordering of the curves is monotone in width. Second, and more importantly, the eigenvalue spectrum in the right panel spans many orders of magnitude and decays precipitously: only a handful of eigenvalues carry appreciable weight before the spectrum collapses toward the numerical floor. By Equation (2.63), the time to resolve an eigendirection is inversely proportional to its eigenvalue, so this steep decay means that all but the leading, smooth eigendirections are learned prohibitively slowly. This is precisely the eigenvalue disparity that Wang, Yu, et al. (2022) identify as the source of spectral bias, and it sets the expectation for everything that follows: the PINN should acquire the low-frequency structure of the solution readily and stall on the high-frequency detail.

4.3 Spectral Bias Along Training

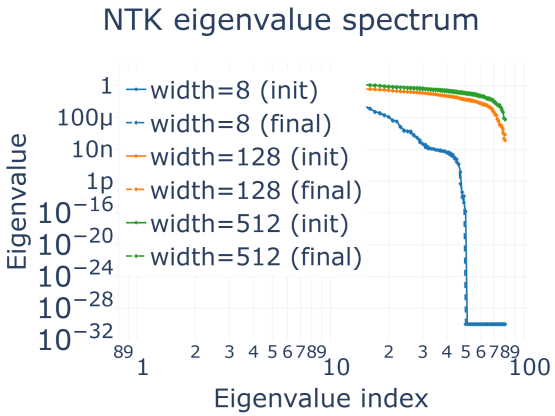
Figure 4.2 tracks the relative spectral error in the low, mid and high wavenumber bands of Equation (3.10) as training proceeds, for all five models. Read within each panel, the figure tells a consistent story; read across panels, it must be interpreted with the caveat of Section 3.3.1, namely that the operators are monitored on the near-linear



(a) Relative parameter drift $\|\theta - \theta_0\|/\|\theta_0\|$.



(b) Relative kernel drift $\|K - K_0\|/\|K_0\|$.



(c) Eigenvalue spectrum of K: initialization (solid) versus end of training (dashed), on log–log axes.

Figure 4.1 – Neural Tangent Kernel diagnostic for the Boussinesq PINN residual at $\alpha = \beta = 3.21$, for widths 8, 128 and 512.

Source: The author (2026).

sample $\alpha = \beta = 0.1$ while the PINNs are monitored on the harder $\alpha = \beta = 3.21$. Cross-family magnitudes are therefore not directly comparable in this figure; the apples-to-apples cross-family comparison is deferred to Section 4.4, where every model is evaluated at the common parameter 3.21.

The within-panel ordering is unambiguous and universal: in all five cases the low band is driven lowest, the mid band trails, and the high band is learned last and least. *No architecture and no regime escapes this ordering*, which is the empirical face of the NTK spectrum of Figure 4.1. What differs is the magnitude of the gap and how far each band is driven down.

The comparisons that *are* valid in this figure — same architecture, same reference sample, differing only in the data term — are decisive. For the PINN at 3.21, adding data lowers the terminal low-band error from about 0.16 to about 0.06 and the mid-band error from about 0.33 to about 0.18, while leaving the high band essentially untouched near 0.70. For the PINO at 0.1, the effect is far larger: the data term drives the low band from about 0.27 down to below 10^{-2} and the mid band from about 0.62 to about 0.02, again with a much smaller improvement in the high band, which falls only from about 0.81 to about 0.45. In both families the lesson is the same: **the supervised term is a powerful lever on the low and mid bands and a weak one on the high band.** The physics residual alone, in the two no-data panels, never brings any band close to the levels reached with data; the PINO (no data) high band barely moves over the entire run.

The FNO panel, monitored at the same easy parameter as the PINO, reaches the lowest low- and mid-band errors of the whole figure (about 0.013 and 0.026) and the lowest high-band error as well (about 0.28). Even so, its high band sits two orders of magnitude above its low band. The single most honest summary of Figure 4.2 is therefore not that operators "solve" spectral bias but that they *compress* it: the best model still leaves a high-frequency residual of roughly 28%, and every other model leaves more.

4.4 Spectral Fidelity at the Intermediate Regime

To compare the families fairly, we now fix the parameter at the intermediate value $\alpha = \beta = 3.21$ and inspect the full spatial spectrum of the prediction at three times, $t = 0$, $t = T/2 = 15$ and $t = T = 30$. Figures 4.3–4.7 report the five models; in every panel the black curve is the reference spatial spectrum and the dashed orange curve is the prediction, with the accompanying relative-error curves and the time-averaged error at the foot of each figure.

The two PINN panels expose the pathology in its clearest form. In the no-data case (Figure 4.3) the deficiency is visible already at $t = 0$: the network cannot even represent the initial sech^2 profile at high wavenumber, and the relative error climbs monotonically

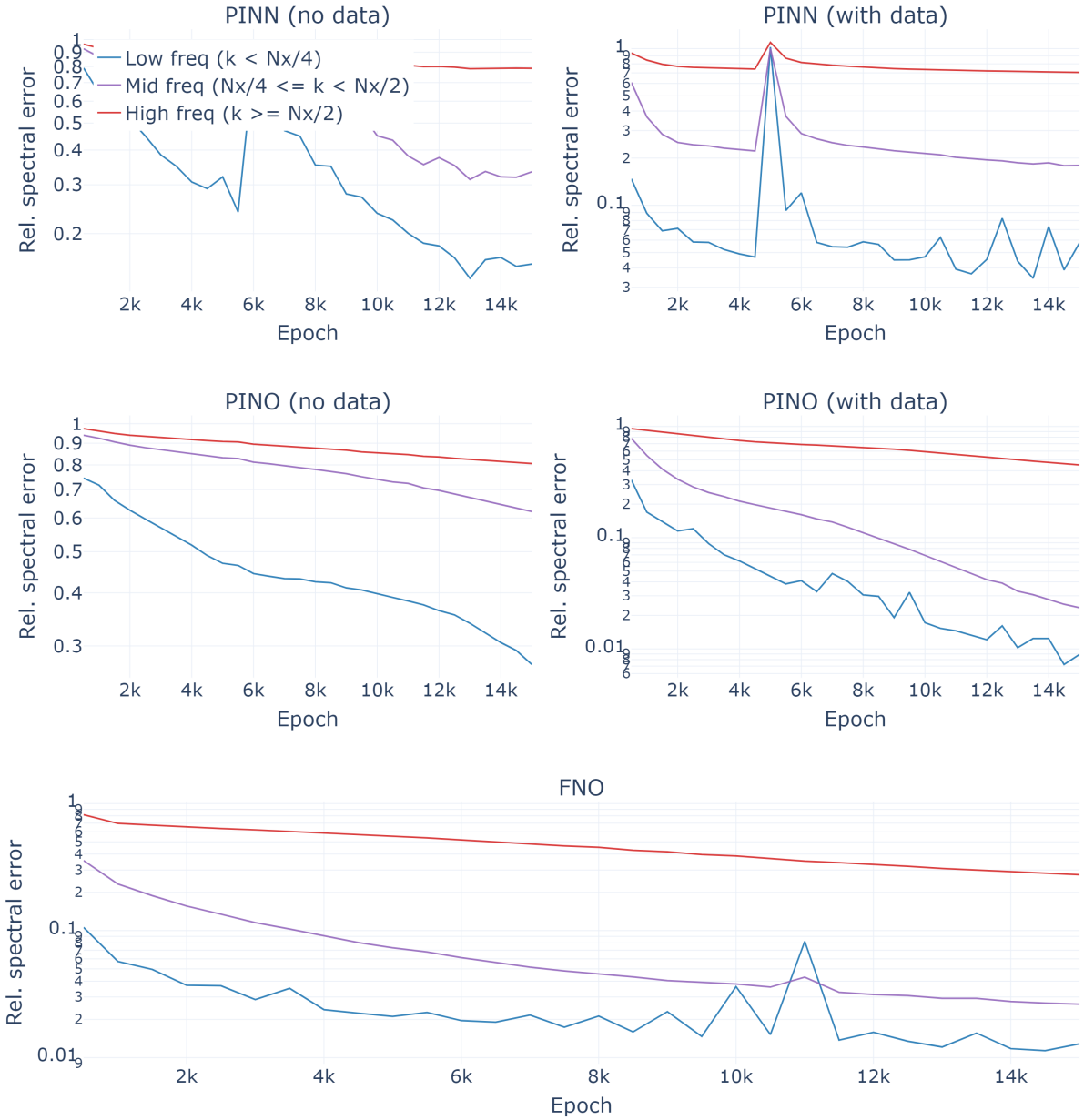


Figure 4.2 – Relative spectral error by frequency band during training. In every panel the low band (blue) is learned first and to the lowest error, the mid band (purple) trails it, and the high band (red) is learned last and least — the signature of spectral bias. Note the mixed reference samples: operators are tracked at $\alpha = \beta = 0.1$, PINNs at $\alpha = \beta = 3.21$.

Source: The author (2026).

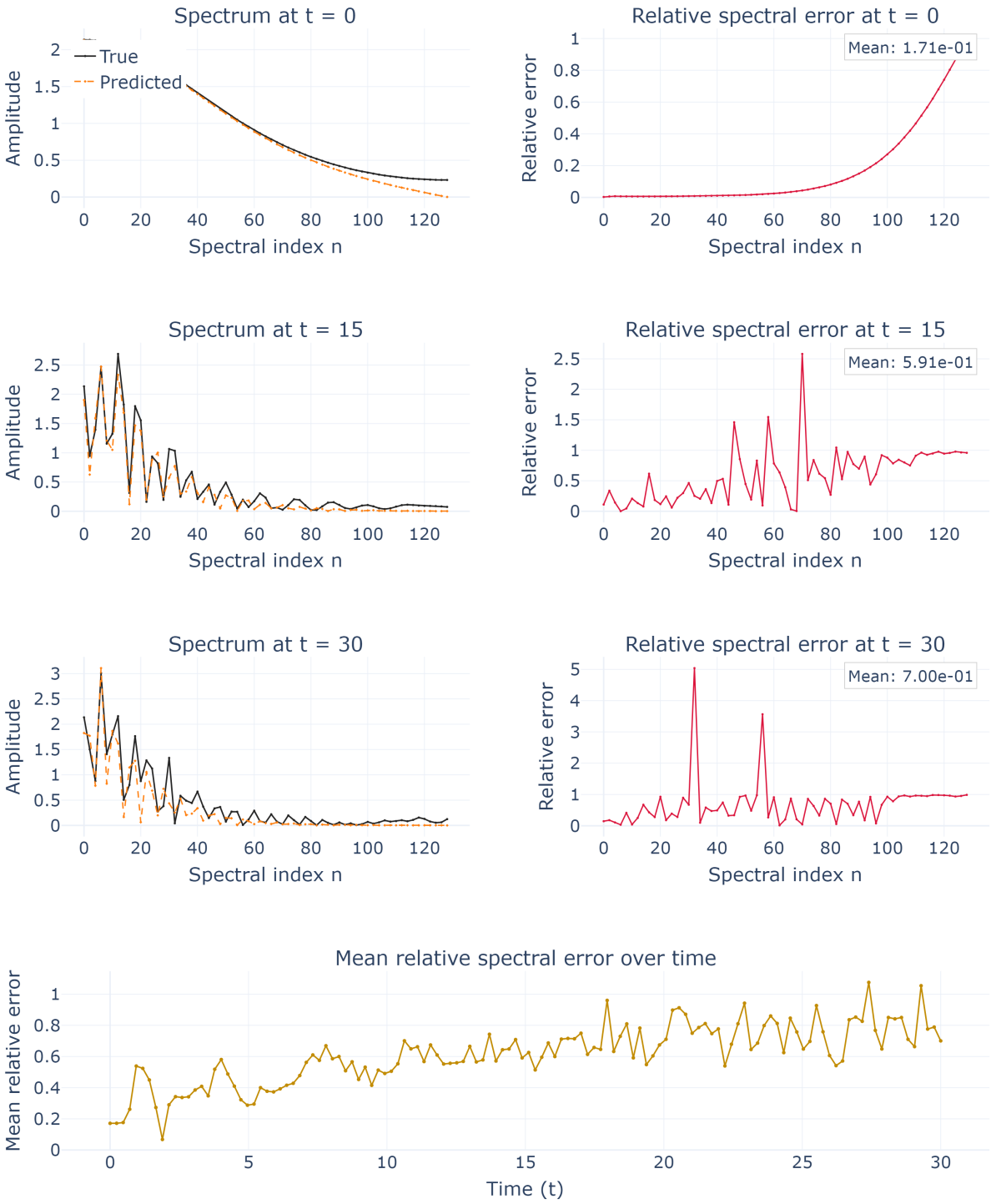


Figure 4.3 – PINN (no data) spatial spectrum at $\alpha = \beta = 3.21$. Already at $t = 0$ the predicted spectrum undershoots the reference tail, and the relative error rises monotonically with wavenumber; the mean spectral error grows from 0.17 at $t = 0$ to 0.70 at $t = 30$.

Source: The author (2026).

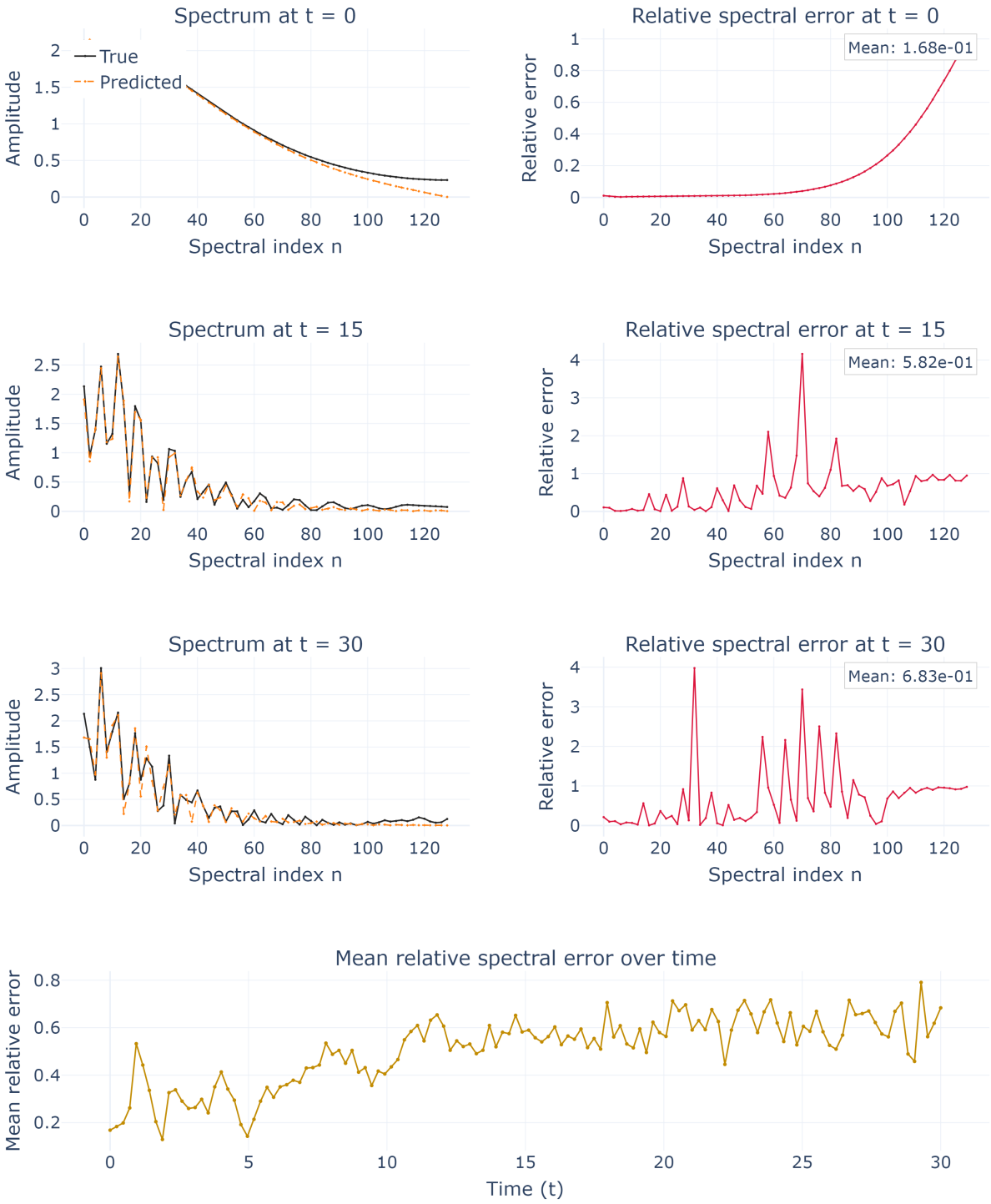


Figure 4.4 – PINN (with data) spatial spectrum at $\alpha = \beta = 3.21$. The low-wavenumber crests are tracked more closely than in Figure 4.3, but the high-wavenumber undershoot and the growth of the mean error in time ($0.17 \to 0.58 \to 0.68$) persist.

Source: The author (2026).

toward unity as the mode index increases. By $t = 30$ the predicted spectrum has lost most of its mid- and high-frequency amplitude, and the mean spectral error reaches 0.70. Adding data (Figure 4.4) tightens the low-wavenumber crests noticeably — consistent with the low-band improvement of Figure 4.2 — yet the aggregate mean error barely changes ($0.17 \rightarrow 0.58 \rightarrow 0.68$), because it is dominated by the mid and high modes that the data term fails to recover at this hard parameter. Data helps the PINN where the NTK is favourable and does not rescue it where the NTK is adverse.

The operators, learning directly in the frequency domain, reverse the picture at low and mid wavenumbers. The FNO (Figure 4.5) reproduces the low and mid spectrum almost exactly at early times — a mean error of 0.013 at $t = 0$, an order of magnitude below the PINN — because those are precisely the modes its spectral layers parametrize. Its weakness is elsewhere: the error concentrates in the high modes and, crucially, *grows monotonically in time*, from a mean of 0.013 at $t = 0$ to 0.40 at $t = 30$. The PINO with data (Figure 4.6) inherits the operator's spectral accuracy but does not share this temporal degradation: its mean error rises only from 0.077 to 0.20 over the same horizon, remaining the most stable of all five models at late times. The contrast is sharp at $t = 30$, where the PINO with data (0.20) is markedly better than the FNO (0.40), even though the FNO was better at $t = 0$; the physics residual trades a slightly worse start for a much steadier trajectory. Finally, the PINO without data (Figure 4.7) forfeits this advantage: stripped of the supervised term, its low- and mid-band undershoot returns and its mean error climbs to 0.33–0.39, midway between the operators-with-data and the PINNs. The physics constraint, on its own, is not a substitute for data.

4.5 The Temporal-Boundary Artifact

The temporal growth of the FNO error observed in Figure 4.5 has a specific and instructive cause. As explained in Section 3.5, the operator applies the Fourier transform along the temporal axis as well as the spatial one, implicitly assuming periodicity in time. Because the Boussinesq trajectory is not periodic — the field at $t = T$ does not match the field at $t = 0$ — this assumption creates an artificial discontinuity at the temporal boundary and excites the Gibbs phenomenon there (GIBBS, 1899). Figures 4.8 and 4.9 make the effect and its remedy visible.

In the FNO panel (Figure 4.8) every parameter value shows the same feature: after an initial transient the time-resolved error settles to a plateau and then jumps sharply at $t = 30$, reaching values two to three times the plateau level. This end-time spike is present at all three nonlinearities and is the dominant contribution to the FNO's late-time error. In the PINO panel (Figure 4.9) the predicted surfaces are visually identical, yet the spike is gone: the error curve rises only slightly toward the boundary. The explanation is that the

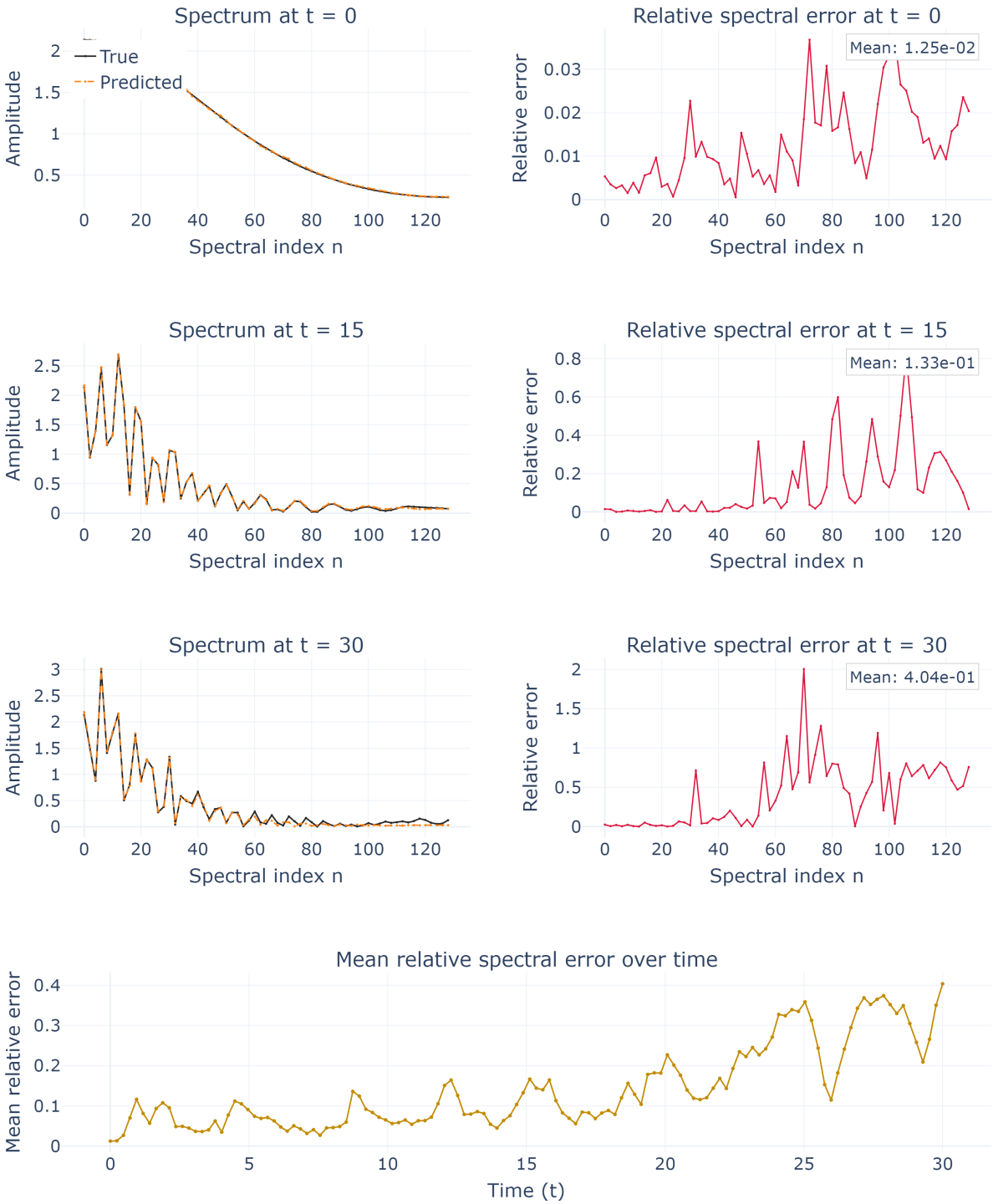


Figure 4.5 – FNO spatial spectrum at $\alpha = \beta = 3.21$. Low and mid wavenumbers are tracked almost exactly (mean error 0.013 at $t = 0$), but the error migrates to the high modes and grows steadily in time, reaching a mean of 0.40 at $t = 30$.

Source: The author (2026).

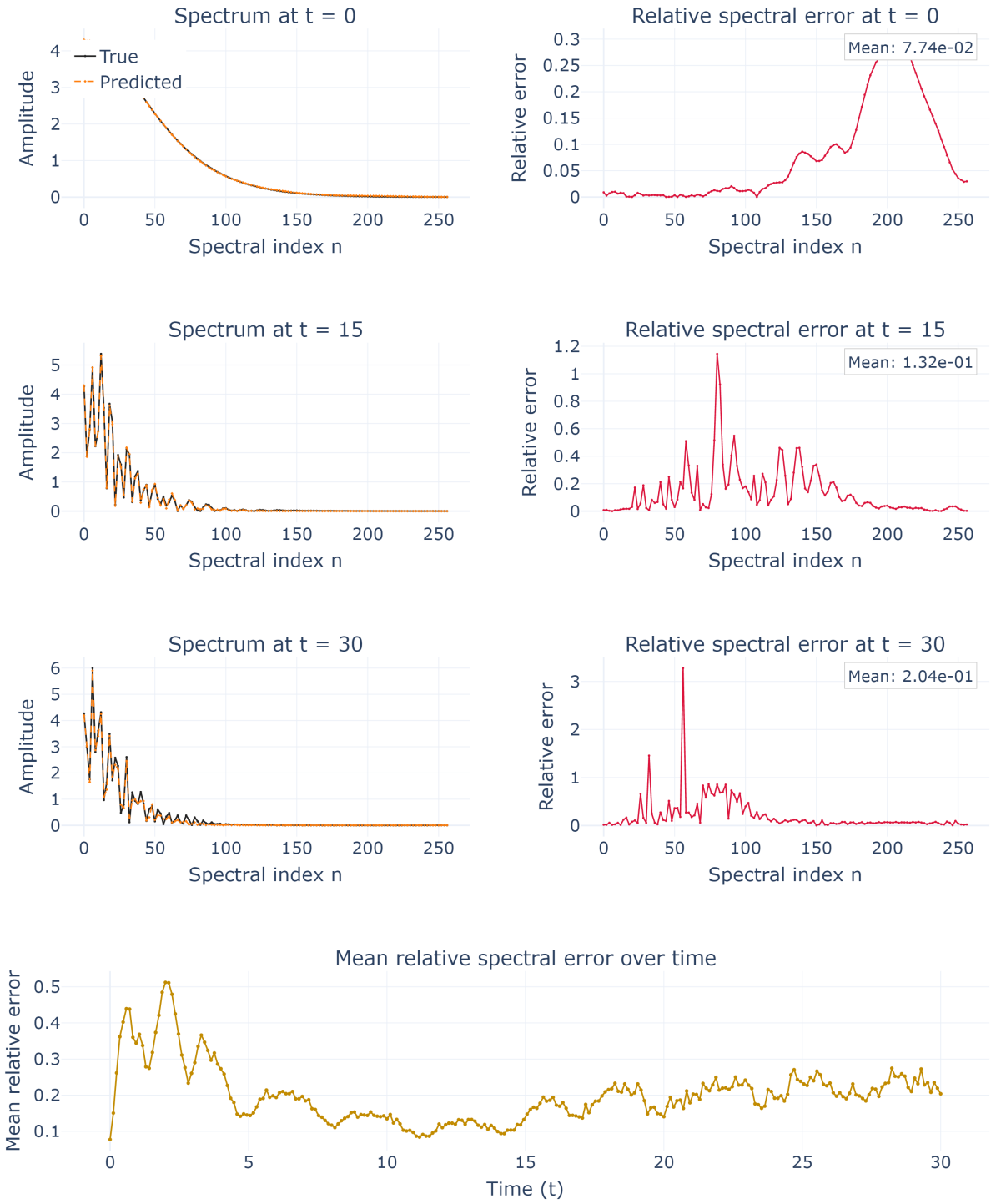


Figure 4.6 – PINO (with data) spatial spectrum at $\alpha = \beta = 3.21$. The prediction tracks the reference across the band and, unlike the FNO, the mean error remains bounded in time ($0.077 \rightarrow 0.13 \rightarrow 0.20$): the physics residual stabilizes the temporal evolution.

Source: The author (2026).

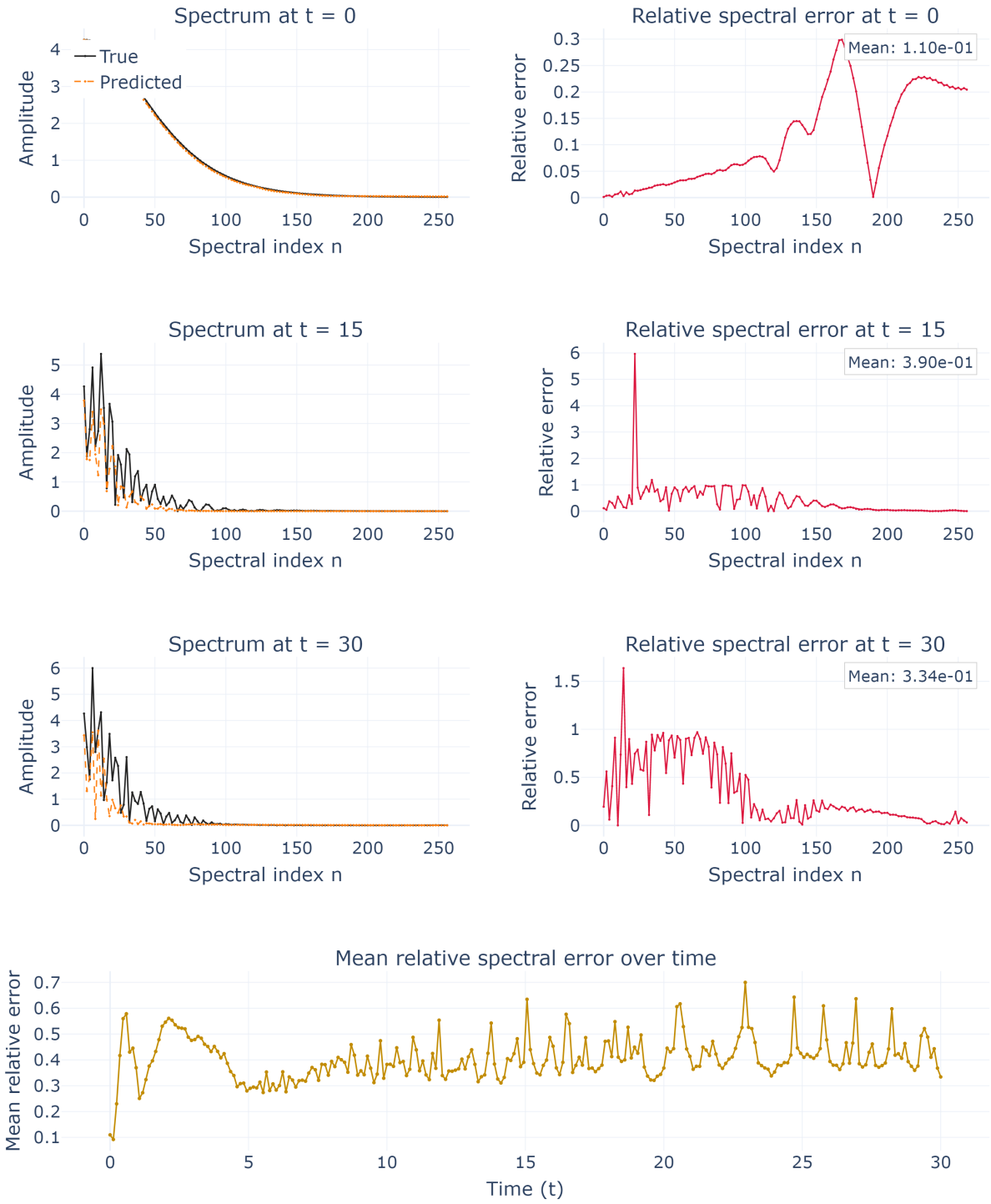


Figure 4.7 – PINO (no data) spatial spectrum at $\alpha = \beta = 3.21$. Without the supervised term the low- and mid-band undershoot returns (mean error 0.11 $\rightarrow$ 0.39 $\rightarrow$ 0.33), confirming that the physics residual alone does not overcome spectral bias.

Source: The author (2026).

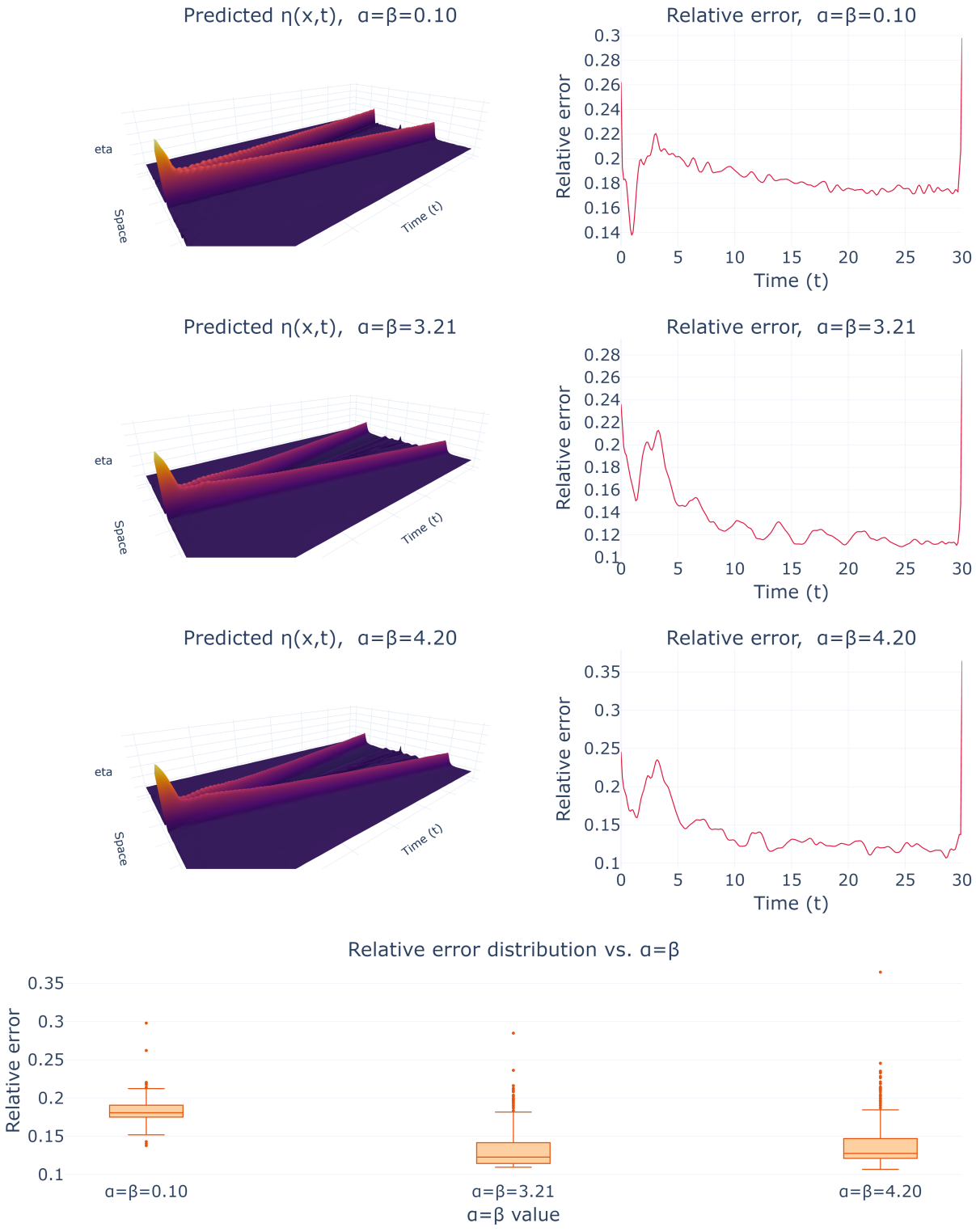


Figure 4.8 – FNO across the nonlinearity axis ($\alpha = \beta \in \{0.1, 3.21, 4.2\}$), evaluated at resolution 256. The predicted surfaces (left) capture the two counter-propagating pulses, but the time-resolved relative error (right) exhibits a pronounced spike at the final time $t = 30$: the temporal Gibbs artifact from treating time as a periodic axis.

Source: The author (2026).

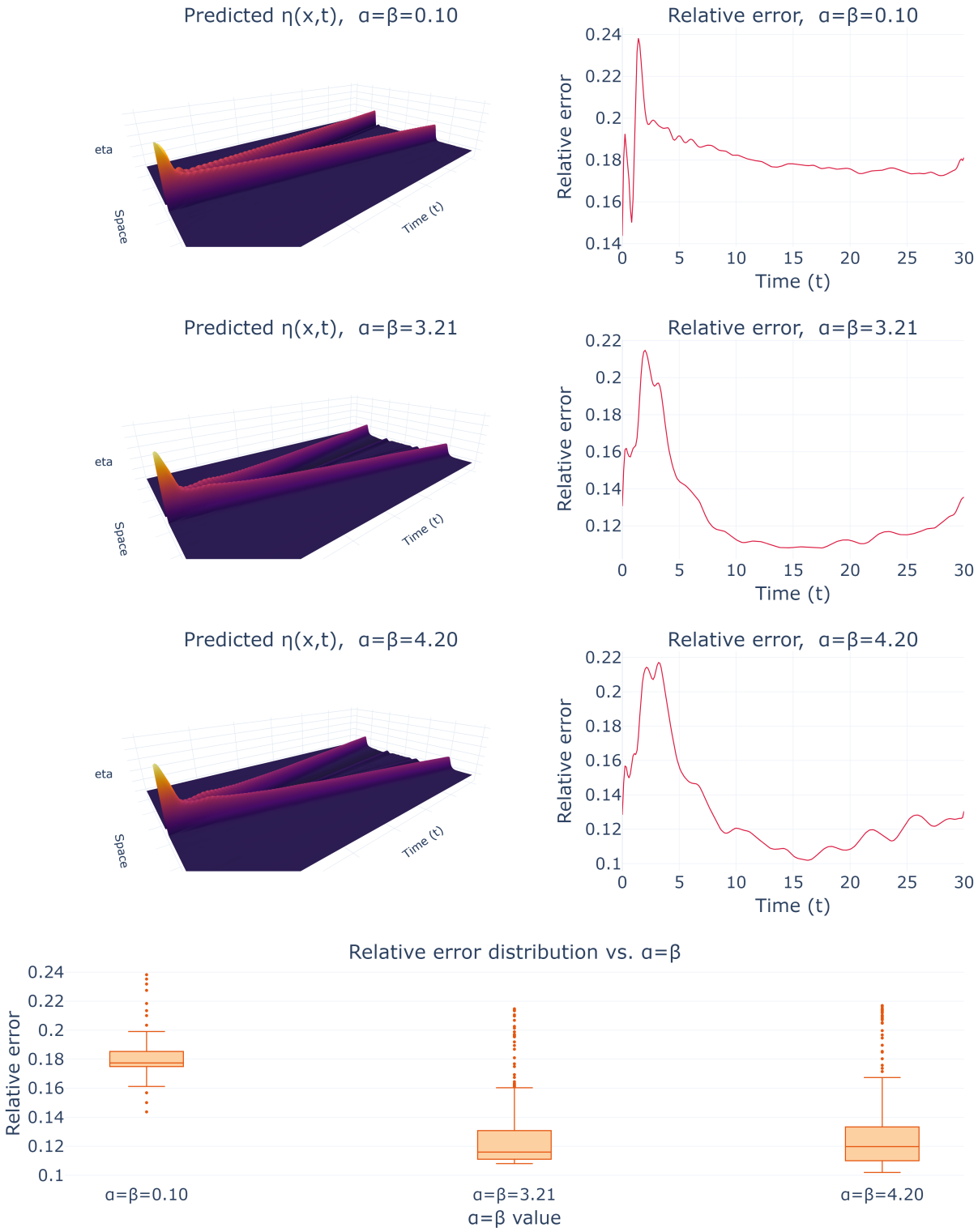


Figure 4.9 – PINO (with data) across the same nonlinearity axis and resolution. The predicted surfaces are visually indistinguishable from the FNO, but the terminal-time spike of Figure 4.8 is absent: the error rises only gently toward $t = 30$. The physics residual and initial-condition term act as a temporal regularizer.

Source: The author (2026).

PDE residual and the initial-condition term penalize exactly the temporal inconsistency that the periodic transform would otherwise tolerate, damping the boundary oscillation. This confirms, and explains, an observation that motivated the experiment: the Gibbs artifact of the FNO is suppressed once the physics is added. It is, however, a suppression and not an elimination — a small terminal rise remains — and it comes at the modelling cost of the residual machinery.

4.6 Accuracy Across the Nonlinearity Axis

The box plots at the foot of Figures 4.8 and 4.9 summarize the distribution of the time-resolved error (3.8) over the whole trajectory, at each of the three parameters. Their most striking feature is counterintuitive: the *smallest* nonlinearity, $\alpha = \beta = 0.1$, carries the *largest* relative error for both operators — a median near 0.18, against roughly 0.12–0.13 at the more nonlinear values 3.21 and 4.2. The strongly nonlinear regime, which one might expect to be hardest, is in fact reproduced with the lowest relative error.

We resist a triumphant reading of this and offer a measured one. Two effects plausibly combine. First, $\alpha = \beta = 0.1$ sits at the extreme lower edge of the training grid (3.2), where the operator has the least surrounding data to interpolate from, so a boundary-of-range penalty is expected. Second, and more mechanically, the relative metric (3.8) normalizes by the instantaneous norm of the reference: in the near-linear regime the solution is dominated by the two clean, slowly deforming pulses of the wave-equation limit, whose energy is concentrated and whose norm is modest, so a fixed absolute error translates into a larger relative one. The nonlinear cases spread more energy across the domain and are, in relative terms, more forgiving. What the box plots do *not* support is the claim that these operators become uniformly more accurate as nonlinearity grows in any absolute sense; they show only that the relative error, under this normalization and this training grid, is non-monotone in the parameter, and that the near-linear boundary case is the least well served.

4.7 Zero-Shot Resolution Behaviour

A central promise of neural operators is discretization-invariance: an operator trained at one resolution can be queried at another without retraining. Figure 4.10 tests this for the PINO with data, trained at resolution 128 and evaluated at 128, 256 and 512 at the fixed parameter $\alpha = \beta = 3.21$.

The qualitative promise holds: at every resolution the operator returns a smooth, coherent surface with the correct pulse structure, and it does so without any retraining or architectural change — the input is simply re-sampled on the finer grid. The quantitative promise is more nuanced. The box plot shows the relative error *increasing* with resolution,

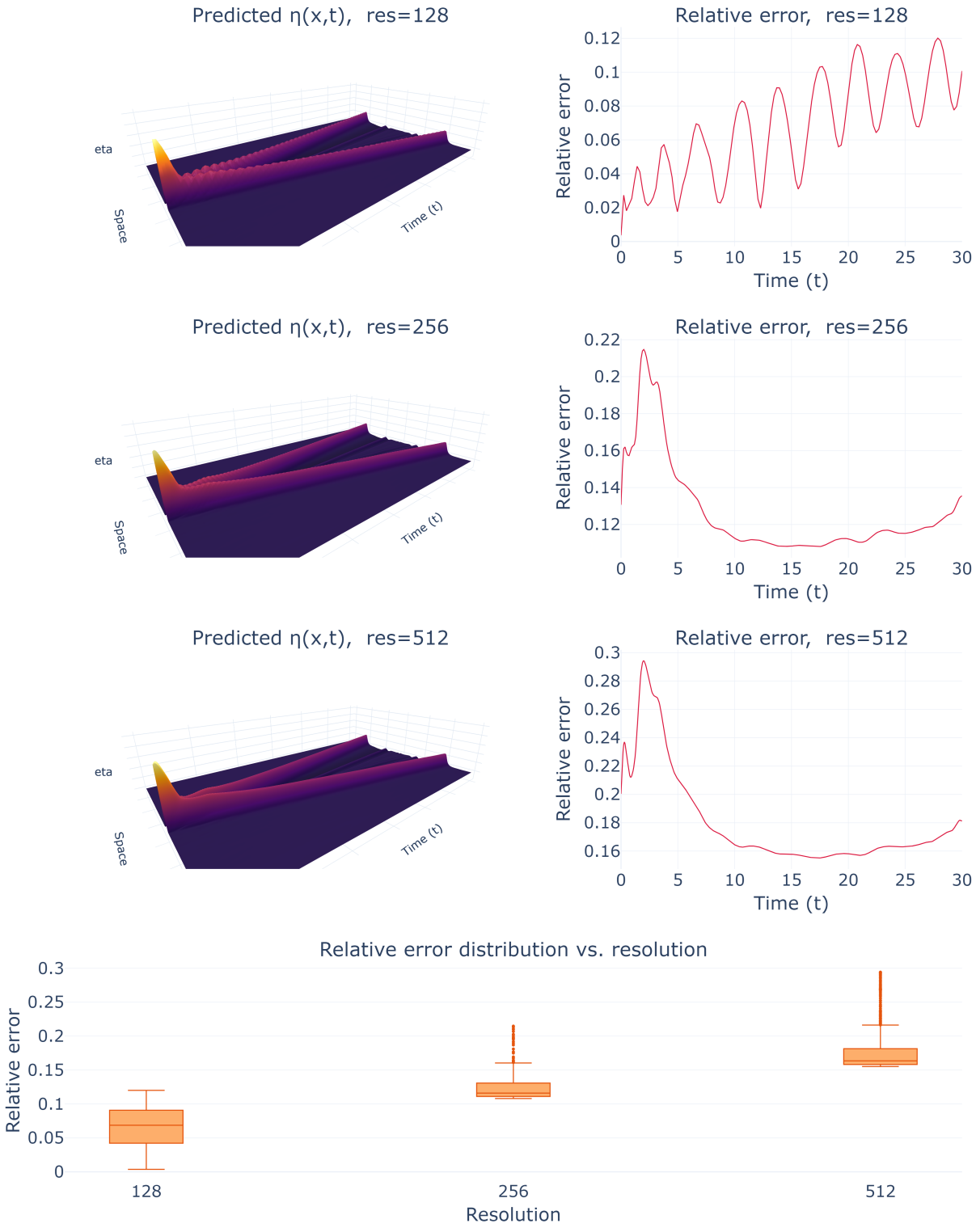


Figure 4.10 – PINO (with data) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3.21$. The operator produces a coherent field at every resolution, but the relative error against the pseudospectral reference *grows* with resolution (median error rising from about 0.07 at 128 to about 0.16 at 512): discretization-invariance of form is not accuracy-invariance.

Source: The author (2026).

from a median near 0.07 at the training resolution of 128 to about 0.12 at 256 and 0.16 at 512. The reason is that the finer references contain genuine high-wavenumber content that the operator, trained at 128 and retaining only 16 Fourier modes per axis, never learned to produce; querying it against a sharper reference simply exposes the fine scales it is missing. There is also a secondary effect at the training resolution itself, where the error oscillates in time in step with the wave reflections, an aliasing signature of the coarse grid. The honest conclusion is that zero-shot superresolution is real as a *geometric* property — the operator accepts and returns fields at any resolution — but not as an *accuracy* property: fidelity is anchored to the spectral band seen in training, and evaluating on a richer grid can only reveal, not recover, the modes left out.

4.8 Synthesis

Taken together, the experiments resist any single-winner narrative and instead map out where each method succeeds and where it fails.

- **Spectral bias is universal.** Every architecture, in every regime, learns low wavenumbers first and best and high wavenumbers last and least (Figures 4.1, 4.2). No method drives the high band below roughly 0.28 relative error; the best in that band is the plain FNO.
- **Data is the decisive lever, physics is not a substitute for it.** Within both the PINN and the PINO families, adding the supervised term sharply reduces the low- and mid-band error, while the physics-only regimes revert toward the biased baseline (Figures 4.2, 4.6, 4.7). Physics alone neither matches data nor removes the bias.
- **Operators dominate at low and mid wavenumbers; PINNs do not.** At the common parameter 3.21, the FNO and the PINO with data track the low and mid spectrum an order of magnitude more accurately than either PINN (Section 4.4), and cover the whole parameter family from one training, whereas each PINN is bound to a single parameter.
- **Physics buys temporal stability.** The FNO's error grows in time and spikes at the temporal boundary through the Gibbs artifact of its periodic time axis; the PINO's residual suppresses that spike and keeps the late-time error bounded (Section 4.5). This is the clearest concrete benefit of the physics term in this study.
- **The remaining limitations are real.** The near-linear boundary parameter is the least accurate (Section 4.6); zero-shot superresolution degrades in accuracy as the

query grid sharpens (Section 4.7); and the high-frequency band is never resolved by any method. These are not artefacts of a single run but consistent features of the design.

The picture that emerges is the one anticipated in Chapter 1: these methods complement, rather than replace, the pseudospectral solver that defines the accuracy ceiling here. Their value lies in amortizing the cost of the parameter sweep and in the temporal regularization that physics supplies, not in a uniform accuracy advantage — and least of all at the high wavenumbers where the reference solver remains unchallenged.

5 CONCLUSION

This work implemented, from first principles, a pseudospectral Boussinesq solver and three families of Scientific Machine Learning methods — Physics-Informed Neural Networks, Fourier Neural Operators and Physics-Informed Neural Operators — and compared them on a single controlled testbed: the one-dimensional Boussinesq system with a fixed solitary initial condition and a nonlinearity–dispersion coefficient $\alpha = \beta$ swept from the near-linear to the strongly nonlinear regime. The pseudospectral solution served throughout as both the training reference and the accuracy ceiling. Rather than crown a single method, the study set out to map where each one succeeds and where it fails, and the numerical results of Chapter 4 deliver a picture that is deliberately free of silver bullets.

5.1 Answers to the Research Questions

The three questions posed in Chapter 1 can now be answered directly.

Accuracy relative to the pseudospectral baseline.

No method matched the reference solver, and none did so uniformly across the spectrum. The operators (FNO and PINO with data) reproduced the low and mid wavenumbers an order of magnitude more accurately than the PINNs and recovered the full parameter family from a single training, but every architecture left an irreducible high-frequency residual, never falling below roughly 28% relative error in the high band. Where each method begins to fail is now precisely located: the PINNs fail already in the mid band and at high nonlinearity; the FNO fails in time, through its temporal-boundary artifact; and all methods fail at the finest scales, which remain the exclusive province of the reference solver.

Data, physics, and their combination.

The presence of supervised data was the single most influential factor in the entire study. Within both the PINN and the PINO families, adding a data term sharply reduced

the low- and mid-band error, whereas the purely physics-informed regimes reverted toward the biased baseline. The physics residual was *not* a substitute for data: on its own it neither matched the accuracy of the data-driven regime nor removed the spectral bias. Its distinctive contribution lay elsewhere — in temporal stability, where the PINO's residual suppressed the Gibbs spike that degrades the FNO at the final time. The most effective configuration observed was therefore the hybrid one, data and physics together, which combined the operator's spectral accuracy with the residual's temporal regularization.

Spectral bias across regimes.

Spectral bias proved universal. The Neural Tangent Kernel diagnostic confirmed its mechanism for the Boussinesq PINN — a steeply decaying kernel spectrum in a near-lazy training regime — and the frequency-band tracking confirmed its consequence in every model: low frequencies learned first and best, high frequencies last and least. What the training regime changes is not the presence of the bias but its severity: data compresses it substantially in the low and mid bands, operators compress it further, but nothing examined here eliminates it. This tempers the abstract's suggestion that operator learning is free of the limitation; the honest statement is that operators mitigate spectral bias far more effectively than PINNs, while still exhibiting it at the highest wavenumbers.

5.2 Limitations and Threats to Validity

Several limitations bound the scope of these conclusions and should be weighed before generalizing them.

- **A single initial condition and a coupled parameter.** All experiments used one solitary profile $\eta(x, 0) = \mathrm{sech}^2(x)$ and held $\alpha = \beta$. The operators therefore learned a one-parameter family, not the full four-parameter Boussinesq landscape, and the reported generalization is generalization in that single coordinate only. Decoupling α and β and varying the initial data would test a substantially harder mapping.

- **Time as a spectral axis.** The operator implementation applies the Fourier transform along the temporal axis, treating time as periodic. This is the direct cause of the terminal-time Gibbs artifact analyzed in Section 4.5. An autoregressive or time-marching operator, as suggested by Li et al. (2020), would avoid the periodicity assumption altogether and is the natural next implementation to compare against.

- **Normalization coupling in the physics-only regime.** As noted in Section 3.2.3, the output normalization constants are drawn from the reference dataset, so the "no-data" models are not strictly data-independent: they inherit the output scale

of the reference. A fully data-free study would fix these constants from the initial condition alone.

- **Fixed capacity and truncation.** Every operator retained only 16 Fourier modes per axis and was trained for a fixed budget of 15,000 iterations at a single learning rate. The persistent high-band error is consistent with this truncation, and a mode or capacity sweep would be needed to separate the architectural limit from the optimization limit.

- **Relative-error normalization.** The non-monotone dependence of the error on nonlinearity (Section 4.6) is partly an artefact of normalizing by the instantaneous solution norm. Complementary absolute or energy-based metrics would give a fuller account of accuracy across the parameter range.

5.3 Future Work

The results point to several concrete continuations. The most immediate is to attack the shared failure mode directly: since spectral bias survives every regime tried here, the spectrum-aware and adaptive-weighting strategies that have been shown to mitigate it — adaptive loss balancing (WANG; TENG, et al., 2021), Fourier feature embeddings, and the second-order and spectrum-aware optimizers reported by Khodakarami et al. (2026) — are the natural levers to test on this same benchmark. A second direction is architectural: a time-marching FNO would remove the temporal Gibbs artifact by construction, isolating how much of the PINO's late-time advantage is intrinsic to the physics term rather than a repair of a periodicity assumption. A third is to broaden the problem: decoupling α and β, varying the initial data, and extending to the two-dimensional Boussinesq system would test whether the ordering of methods observed here is stable under a genuinely higher-dimensional solution operator. Finally, a comparison against DeepONet (LU et al., 2021) and against hybrid schemes that hand the high-frequency residual back to a classical corrector would probe whether the complementary — rather than competitive — role identified for these methods can be turned into a practical solver that is both fast and spectrally faithful.

The overarching message is unchanged from the outset: for the nonlinear Boussinesq system, Scientific Machine Learning methods are best understood as complements to classical solvers. They amortize the cost of a parameter sweep and, when physics is added, gain temporal stability; but at the fine scales that make dispersive wave problems difficult, the pseudospectral reference remains, for now, unrivalled.

REFERENCES

ARORA, S. et al. Fine-Grained Analysis of Optimization and Generalization for Overparameterized Two-Layer Neural Networks. In: PROCEEDINGS of the 36th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2019. v. 97, p. 322–332.

BAKER, N. et al. **Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence**. Washington, DC, 2019. DOI: 10.2172/1478744.

BIHLO, A.; POPOVYCH, R. O. Physics-informed neural networks for the shallow-water equations on the sphere. **Journal of Computational Physics**, 2022. Available from: <https://arxiv.org/abs/2104.00615>.

BONA, J. L.; CHEN, M.; SAUT, J.-C. Boussinesq equations and other systems for small-amplitude long waves in nonlinear dispersive media: I. Derivation and linear theory. **Journal of Nonlinear Science**, v. 12, p. 283–318, 2002. DOI: 10.1007/s00332-002-0466-4.

BONEV, B. et al. Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere. In: INTERNATIONAL Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Available from: <https://arxiv.org/abs/2306.03838>.

BORDELON, B.; CANATAR, A.; PEHLEVAN, C. Spectrum Dependent Learning Curves in Kernel Regression and Wide Neural Networks. In: PROCEEDINGS of the 37th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2020. Available from: <https://arxiv.org/abs/2002.02561>.

BOUSSINESQ, J. Théorie des ondes et des remous qui se propagent le long d'un canal rectangulaire horizontal. **Journal de Mathématiques Pures et Appliquées**, v. 17, p. 55–108, 1872.

BRUNTON, S. L.; PROCTOR, J. L.; KUTZ, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. **Proceedings of the National Academy of Sciences**, v. 113, n. 15, p. 3932–3937, 2016. DOI: 10.1073/pnas.1517384113.

BURDEN, R. L.; FAIRES, J. D. **Numerical Analysis**. 9. ed. Boston: Brooks/Cole, 2011.

CAO, Y. et al. Towards Understanding the Spectral Bias of Deep Learning. In: PROCEEDINGS of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/1912.01198>.

CHEN, R. T. Q. et al. Neural Ordinary Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07366>.

CHEN, T.; CHEN, H. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. **IEEE Transactions on Neural Networks**, v. 6, n. 4, p. 911–917, 1995.

CUOMO, S. et al. Physics-informed neural networks for data-driven simulation: Advantages, limitations, and opportunities. **Physica A: Statistical Mechanics and its Applications**, 2023. DOI: 10.1016/j.physa.2022.128415.

CYBENKO, G. Approximation by superpositions of a sigmoidal function. **Mathematics of Control, Signals and Systems**, v. 2, n. 4, p. 303–314, 1989.

DE PINA, I. et al. Investigating Steady Unconfined Groundwater Flow using Physics Informed Neural Networks. **Advances in Water Resources**, 2023. DOI: 10.1016/j.advwatres.2023.104445.

_____. Investigating Steady Unconfined Groundwater Flow Using Physics-Informed Neural Networks. **arXiv preprint arXiv:2112.13792**, 2021. Available from: <https://arxiv.org/abs/2112.13792>.

DEBNATH, L. **Nonlinear Partial Differential Equations for Scientists and Engineers**. 3. ed. New York: Birkhäuser, 2012.

ENGSIG-KARUP, A. P. et al. Fourier pseudospectral methods for 2D Boussinesq-type equations. **Ocean Modelling**, 2012. DOI: 10.1016/j.ocemod.2012.05.003.

GIBBS, J. Willard. Fourier's Series. **Nature**, v. 59, n. 1539, p. 606, 1899. DOI: 10.1038/059606a0.

GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

GUPTA, G. et al. Multiwavelet-based Operator Learning for Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2109.13459>.

HORNIK, K.; STINCHCOMBE, M.; WHITE, H. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989.

JACOT, A.; GABRIEL, F.; HONGLER, C. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07572>.

JAGTAP, A. D. et al. Deep Learning of Inverse Water Waves Problems Using Multi-Fidelity Data: Application to Serre–Green–Naghdi Equations. **arXiv preprint arXiv:2202.02899**, 2022. Available from: <https://arxiv.org/abs/2202.02899>.

KARNIADAKIS, G. E. et al. Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021. DOI: 10.1038/s42254-021-00314-5.

KHODAKARAMI, S. et al. Spectral bias in physics-informed and operator learning: analysis and mitigation guidelines. **arXiv preprint**, 2026. Available from: <https://arxiv.org/abs/2602.19265>.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.

KREYSZIG, E. **Introductory functional analysis with applications**. [S.l.]: John Wiley & Sons, 1978.

KRISHNAPRIYAN, A. S. et al. Characterizing Possible Failure Modes in Physics-Informed Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. v. 34, p. 26548–26560.

KUMAR, S.; DHAMA, S. Physics-informed neural networks for exploring soliton collisions in the non-integrable Schamel equation in plasmas. **Physics of Fluids**, v. 37, n. 1, 2025. DOI: 10.1063/5.0301334.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, 2015. DOI: 10.1038/nature14539.

LI, Z. et al. Fourier Neural Operator for parametric partial differential equations. In: INTERNATIONAL Conference on Learning Representations (ICLR). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2010.08895>.

_____. Physics-Informed Neural Operator for Learning Partial Differential Equations. **ACM/IMS Transactions on Data Science**, 2023. Available from: <https://arxiv.org/abs/2111.03794>.

_____. **PINO: Physics-Informed Neural Operator – Reference Implementation**. [S.l.: s.n.], 2023. GitHub repository. Available from: <https://github.com/neuraloperator/PINO>.

LI, Z. et al. Fourier neural operator for parametric partial differential equations. **arXiv preprint arXiv:2010.08895**, 2020.

LINNAINMAA, A. Taylor expansion of the accumulated rounding error. **BIT Numerical Mathematics**, v. 10, n. 1, p. 47–55, 1970.

LKHAGVASUREN, B.; CHOI, B.-J.; JIN, H. S. Applications of the Fourier neural operator in a regional ocean modeling and prediction. **Frontiers in Marine Science**, v. 11, 2024. DOI: 10.3389/fmars.2024.1383997.

LU, L. et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. **Nature Machine Intelligence**, v. 3, p. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5.

MARTINS, L. G. **Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo**. 2023. MA thesis – Universidade Federal do Paraná, Curitiba.

MARTINS, L. G.; FLAMARION, M. V.; RIBEIRO-JR, R. A forced Boussinesq model with a sponge layer. **Partial Differential Equations in Applied Mathematics**, v. 10, p. 100661, 2024. DOI: 10.1016/j.padiff.2024.100661.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **Bulletin of Mathematical Biophysics**, v. 5, n. 4, p. 115–133, 1943.

NOCEDAL, J. Updating quasi-Newton matrices with limited storage. **Mathematics of Computation**, v. 35, n. 151, p. 773–782, 1980.

PARIKH, N.; BOYD, S. Proximal Algorithms. **Foundations and Trends in Optimization**, v. 1, n. 3, p. 123–231, 2013. DOI: 10.1561/2400000003.

PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: ANNUAL Conference on Neural Information Processing Systems (NeurIPS). Vancouver: [s.n.], 2019. p. 8024–8035. Available from: <https://arxiv.org/abs/1912.01703>.

PATEL, A. et al. A Neural Operator-Based Emulator for Regional Shallow Water Dynamics. **arXiv preprint**, 2025. Available from: <https://arxiv.org/abs/2502.14782>.

PEI, K. et al. Data-driven soliton mappings for integrable fractional nonlinear wave equations via deep learning with Fourier neural operator. **Chaos, Solitons & Fractals**, v. 165, 2022. DOI: 10.1016/j.chaos.2022.112848.

PEREGRINE, D. H. Long waves on a beach. **Journal of Fluid Mechanics**, v. 27, n. 4, p. 815–827, 1967. DOI: 10.1017/S0022112067002605.

RACKAUCKAS, C. et al. Universal Differential Equations for Scientific Machine Learning. **arXiv preprint arXiv:2001.04385**, 2020.

RAHAMAN, N. et al. On the spectral bias of neural networks. In: PMLR. INTERNATIONAL Conference on Machine Learning (ICML). Long Beach: [s.n.], 2019. p. 5301–5310.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **arXiv preprint arXiv:1711.10561**, 2017.

_____._____. **Journal of Computational Physics**, v. 378, p. 686–707, 2019.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, n. 6, p. 386–408, 1958.

ROSOFSKY, S. et al. Applications of Physics-Informed Neural Operators (PINOs): Shallow Water Equations and Beyond. **arXiv preprint**, 2023. Available from: <https://github.com/shawnrosofsky/PINO_Applications>.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, n. 6088, p. 533–536, 1986.

STRANG, G. **Linear Algebra and Its Applications**. 4. ed. Belmont: Thomson Brooks/Cole, 2006.

STROGATZ, S. H. **Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering**. 2. ed. Boca Raton: CRC Press, 2018.

SÜLI, E.; MAYERS, D. F. **An Introduction to Numerical Analysis**. Cambridge: Cambridge University Press, 2003.

TODO. A Review of Physics-Informed Machine Learning in Fluid Mechanics. **Energies**, v. 16, n. 5, 2023. DOI: 10.3390/en16052343.

TODO. Modelo Numérico de Boussinesq Adaptado para Ondas de Embarcação. **Revista Brasileira de Recursos Hídricos**, v. 15, n. 4, p. 5–15, 2010. DOI: 10.21168/rbrh.v15n4.p5-15.

TREFETHEN, L. N. **Spectral Methods in MATLAB**. Philadelphia: Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719598.

UCHIDA, T. et al. Ocean Emulation With Fourier Neural Operators: Double Gyre. **Journal of Advances in Modeling Earth Systems**, v. 17, n. 1, 2025. DOI: 10.1029/2023MS004137.

WANG, L. et al. Explorations of nonlinear waves of the Boussinesq and Camassa-Holm equations using PINNs. **Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences**, v. 480, n. 2286, 2024. DOI: 10.1098/rspa.2023.0580.

WANG, S.; TENG, Y.; PERDIKARIS, P. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. **SIAM Journal on Scientific Computing**, v. 43, n. 5, a3055–a3081, 2021. DOI: 10.1137/20M1318043.

WANG, S.; YU, X.; PERDIKARIS, P. When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective. **Journal of Computational Physics**, v. 449, p. 110768, 2022. DOI: 10.1016/j.jcp.2021.110768.

WHITHAM, G. B. **Linear and Nonlinear Waves**. New York: Wiley-Interscience, 1974.

XU, Z.-Q. J. et al. Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks. In: 5. COMMUNICATIONS in Computational Physics. [S.l.: s.n.], 2020. v. 28, p. 1746–1767. DOI: 10.4208/cicp.OA-2020-0085.

YU, J. et al. Spherical multigrid neural operator for improving autoregressive global weather forecasting. **Scientific Reports**, v. 15, n. 1, 2025. DOI: 10.1038/s41598-025-96208-y.

ZHANG, Y. et al. Influence of layer and neuron configurations on Boussinesq equation solutions via a bilinear neural network. **Nonlinear Dynamics**, v. 112, p. 12315–12333, 2024. DOI: 10.1007/s11071-024-09708-3.

ZHENG, S. et al. Prediction of phase transition and time-varying dynamics of the (2+1)-dimensional Boussinesq equation by parameter-integrated PINNs with phase domain decomposition. **Physical Review E**, v. 108, n. 4, 2023. DOI: 10.1103/PhysRevE.108.045303.

ZHONG, M.; YAN, Z.; TIAN, S. Data-driven parametric soliton-rogon state transitions for nonlinear wave equations using deep learning with Fourier neural operator. **Communications in Theoretical Physics**, v. 75, n. 2, p. 025001, 2023. DOI: 10.1088/1572-9494/acab55.